



NBA Shot Quality Capstone

Rough Draft

DAT 490 — Data Science Capstone (2026 Summer A)

Team Members

Project Lead / Lit Review & Ethics	Cole Campbell
Modeling Lead	Marc Rajesh
EDA & Visualization Lead	Calder Wyllie
Data Engineer	Germain Meza

Table of Contents

Abstract / Executive Summary	3
Project Plan	4
Review of the Literature	13
Final Research Questions.....	17
Exploratory Data Analysis.....	18
Methodology	26
Analysis.....	30
Data Visualizations	34
Ethical Recommendations	39
Challenges.....	42
Recommendations and Next Steps.....	44
References	45
Appendix: Source Code	46

Abstract / Executive Summary

This project models the quality of NBA shots — the probability that any given field goal attempt goes in based on the context around it — and uses that model to separate two things traditional statistics conflate: Shot selection and shot-making skill. The organization at the center of the work is the National Basketball Association, the premier professional basketball league in North America and the stakeholder our analysis is built for. The league office, its thirty franchises, and its broadcast partners all rely on player evaluation to make decisions about contracts, rosters, and how the game is presented, and a sharper understanding of shot quality speaks directly to that work.

Our data comes from the NBA's official statistics API, which provides shot-level detail for every field goal attempt from the 2013-14 season through the present — roughly two million shots — supplemented by a Kaggle dataset of 2014-15 shot logs that adds the per-shot tracking features (defender distance, shot clock, touch time, and dribbles) needed to model shot difficulty. We trained a gradient-boosted tree model on those features, reaching an accuracy of about 62 percent and a ROC-AUC of about 0.64, comfortably above the no-information baseline, which confirms that shot context is a relevant factor in predicting shot success.

We applied the model to three research questions. First, we found that the players and teams who consistently make more shots than expected span four distinct playstyles — on-ball creators, catch-and-shoot specialists, interior finishers, and mid-range scorers — rather than concentrating in any single type. Second, we found that the league's transformation over the tracking era was driven by shot selection rather than league-wide shooting improvement: Mid-range attempts collapsed by about 17% of total shot share while three-point attempts rose, even though efficiency within each zone stayed largely stable. Third, we found that a shot-quality-adjusted ranking of players differs substantially from traditional field goal percentage (Spearman's ρ of about 0.39), systematically elevating perimeter shooters who make difficult shots and lowering interior players who take easy ones. The overarching conclusion is that shot quality is a measurable and informative lens on player value — one that surfaces contributors that traditional metrics overlook — provided it is used as one input among many rather than one, complete verdict on a player.

Project Plan

Executive Summary

This project models the probability that an NBA shot is made based on the context around it — where it was taken, how much time was on the clock, how close the defender was, who took it, and how the game was going. Our data covers every field goal attempt from the 2013-14 season through the current season, roughly two million shots.

From that model we plan to deliver three things: a ranked list of players and teams who consistently outperform what their shots predict, a longitudinal view of how shot success by court location has changed across the tracking era, and a comparison of shot-quality-adjusted player rankings against traditional efficiency metrics.

Profile of the Organization

The National Basketball Association (NBA) is the premier professional men's basketball league in North America and one of the most globally recognized sports leagues in the world. It is the company at the center of our analysis and the stakeholder our findings serve; the league office, individual franchises, and broadcast partners would all benefit from a sharper understanding of shot quality and player evaluation.

Organization Details

Founded: June 6, 1946 (as the Basketball Association of America); merged with the National Basketball League in 1949 to form the NBA as it exists today.

Headquarters: Olympic Tower, 645 Fifth Avenue, New York, NY 10022, United States.

Structure: 30 franchises, divided into the Eastern and Western Conferences. The league operates as a single business entity (NBA, LLC) with individually owned teams.

Website: www.nba.com

Business Description

The NBA's core product is professional basketball: 30 teams play an 82-game regular season followed by a 16-team postseason that ends in the NBA Finals. Revenue comes from national and regional media rights, ticket sales, sponsorships, merchandising, and growing international business. The league also operates the WNBA, the NBA G League (its developmental league), the Basketball Africa League, and the NBA 2K League (esports).

Beyond the on-court product, the NBA invests heavily in analytics and player tracking. The league installed optical tracking cameras in every arena starting in the 2013-14 season; that decade of shot-by-shot tracking data is the foundation our analysis builds on.



Madison Square Garden — home of the New York Knicks. Photo: Ganley894, public domain via Wikimedia Commons.

Financials

League revenue: approximately \$11 billion annually (2023-24 season) and growing as media-rights deals expand.

Media rights: in 2024 the league signed a new 11-year, ~\$76 billion media-rights agreement with Disney (ABC/ESPN), NBCUniversal, and Amazon Prime Video, which kicked off with the 2025-26 season.

Average franchise valuation: approximately \$4.4 billion as of 2024 (Forbes annual valuations), with top franchises like the Warriors, Knicks, and Lakers valued above \$7 billion each.

Employees: about 1,500 at the league office; teams employ roughly 200-400 staff each, putting total league-wide employment in the range of 10,000-12,000.

Key Executives

Adam Silver — Commissioner (since February 2014)

Mark Tatum — Deputy Commissioner & Chief Operating Officer

Byron Spruell — President, League Operations

Tammy Henault — Chief Marketing Officer

Each of the 30 teams also has its own president of basketball operations, general manager, and analytics staff — the people most likely to use the kind of analysis we're proposing.

Major Competitors

As a professional sports league, the NBA competes for fan attention, broadcast dollars, and athletic talent. The most direct competitors are:

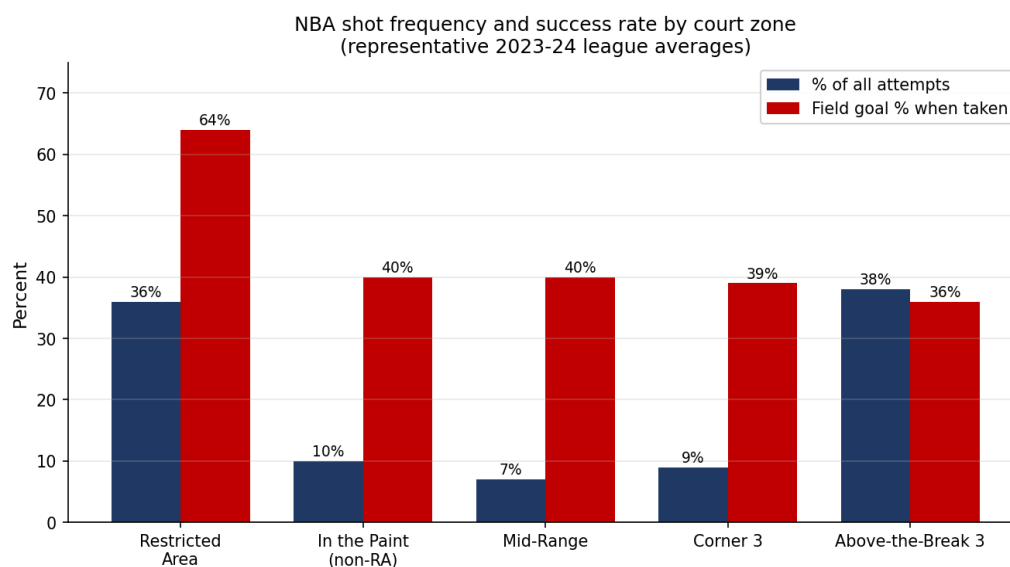
- National Football League (NFL) — the dominant US sports league by revenue and viewership
- Major League Baseball (MLB)
- National Hockey League (NHL)
- Major League Soccer (MLS) — growing quickly in the US market
- International basketball leagues like the EuroLeague and FIBA-sanctioned competitions, especially for talent acquisition
- College basketball (NCAA), especially during March Madness
- Streaming entertainment platforms competing for the same leisure time as live sports

Business / Analysis Opportunity

Since the 2013-14 season, the NBA has captured optical tracking data for every shot taken in every game. That gives us about two million shot attempts with good context — court coordinates, defender distance, shot clock remaining, touch time, dribbles before the shot, and outcome. Combined with player bios, team metadata, and historical performance data going back to 1947, we have a dataset that gives us plenty of features for modeling and visualization and easily clears the project's data requirements.

The opportunity we see is to build an expected shot value model — a way to predict the probability that any given shot goes in based on the context around it — and use that model to separate two things traditional metrics conflate: shot selection and shot-making skill. Two players with the same field goal percentage can be taking very different shots, and our analysis can quantify that difference.

The chart below shows a representative view of how the league distributes its shots and how often each zone goes in. The pattern frames the opportunity: the shots taken most often sit at opposite ends of the success spectrum, and the mid-range — once a staple of the league — has shrunk dramatically.



Representative example. Actual figures will come from our pulled data during EDA.

Research Questions

We've developed three research questions, each with a clear connection to how the NBA — and the teams that make it up — could use the results.

RQ1 — Which players and teams consistently outperform their expected shot value, and what distinguishes them?

Traditional efficiency metrics like field goal percentage credit players for both shot selection (taking easier shots) and shot-making skill (finishing harder shots). Front offices need to tell the two apart when evaluating talent, signing free agents, and building rosters. By identifying players and teams who consistently beat what our model expects, we can give the NBA and its teams a clearer view of who is creating value beyond what the shots themselves warrant.

RQ2 — How has shot success by court location changed across the 2013-present tracking era?

Over the last decade, the NBA has gone through one of the most dramatic strategic shifts in any major sport — the explosion of three-point shooting. This question asks how the geography of effective shooting has changed, and whether the value of mid-range shots, restricted-area finishes, and corner threes has shifted along with team strategy. The league, broadcast partners, and rules committees all benefit from a quantified view of how the game has evolved.

RQ3 — Does shot-quality-adjusted player ranking differ noticeably from traditional efficiency metrics (FG%, eFG%, TS%)?

If our shot-quality-adjusted metric ranks players very similarly to the metrics already in wide use, then the traditional metrics are doing fine, and the added complexity isn't worth much. If the rankings clearly diverge, then teams currently relying on FG%, eFG%, or TS% may be over-or-under-valuing specific players, with direct implications for contracts, trades, and roster construction.

Hypotheses

For each research question, we have an initial expectation of what we'll find. These are forecasts, not conclusions — we'll let the data confirm or push back.

H1 — for RQ1

We expect that players who consistently outperform expected shot value will cluster into a small number of identifiable archetypes — primary shot creators with elite touch, low-usage floor spacers who shoot only when open, and frontcourt players who finish at extremely high rates near the rim. NBA roles are reasonably well-defined, and the players who exceed expectations likely do so for role-specific reasons rather than randomly.

H2 — for RQ2

We expect to see a meaningful decline in mid-range shot success over time, alongside an increase in three-point and restricted-area success rates. As teams shifted attempts away from the mid-range, the shots that remain in that zone are likely taken by the league's best mid-range shooters — but the volume drop and devaluation should still show up as reduced league-wide effectiveness from that area.

H3 — for RQ3

We expect moderate-to-low rank correlation between our shot-quality-adjusted metric and the traditional metrics, particularly for low-usage role players. Traditional metrics reward players who take easy shots (high-percentage layups, wide-open threes), while a shot-quality-adjusted view should give credit to players who consistently make difficult shots. The two views should disagree most for specialists at the extremes of shot difficulty.

Data

Our analysis draws from one primary source and two supporting datasets. All of them map to NBA.com identifiers, so joins are reliable and reproducible.

Primary Source — NBA Stats API

We're pulling shot detail, game logs, players, and teams from the NBA's official statistics API through the open-source `nba_api` Python package. The shot detail covers every field goal attempt from the 2013-14 season through the current season with court coordinates, shot zone, defender distance, shot clock remaining, touch time, dribbles before the shot, and the outcome. Game logs, player bios, and team metadata round out the join context.

Supporting Source — Kaggle: NBA Shot Logs 2014-15

A single-season, pre-cleaned dataset of about 128,000 shots from the 2014-15 season, originally scraped from the same NBA Stats API. We're using this for early prototyping while the full multi-season pull from the primary API runs in the background.

Supporting Source — Kaggle: NBA Stats (1947-present)

Maintained by `sumitrodatta`, this dataset covers NBA, ABA, and BAA stats from 1947 to the present and is actively updated. It gives us long-horizon player history for active players whose careers predate the 2013-14 tracking era — career averages, prior-era context, and career-arc features that the primary API doesn't expose conveniently.

Integration

Every source maps to NBA.com identifiers (`player_id`, `team_id`, `game_id`), so all our joins are ID-based rather than name-based. The product of the data pipeline is one enriched shot-level table — each row is a single shot carrying its own context, the game's context, the shooter's context, and the team's context. That table is what every downstream analysis reads from.

Methodology

RQ1 — Shot-Level Classification, Residual Analysis, and Clustering

RQ1 is layered rather than a single method. The base layer is binary classification at the shot level: we train a model to predict the probability a single shot is made given its context. We'll start with logistic regression as a baseline because of its interpretability, then move to gradient-boosted trees (XGBoost or LightGBM) to capture non-linear interactions between features like defender distance, shot clock, and shot type.

From the trained model, we compute residuals for every player and every team — actual makes minus model-expected makes. Players with consistently positive residuals are our overperformers. To answer the 'what distinguishes them' part of the question, we cluster those overperformers using playing-style features (usage rate, shot mix, position, team pace) to identify archetypes.

RQ2 — Spatial Binning, Smoothed Heat Maps, and Longitudinal Comparison

RQ2 is descriptive rather than predictive. We'll bin the court into standard NBA shot zones (restricted area, in-the-paint non-RA, mid-range, corner three, above-the-break three), compute shot success rate per zone for each season from 2013-14 through the present and visualize using kernel density estimation to produce smooth heat maps. To quantify whether changes between seasons are statistically meaningful, we'll use paired comparisons of zone-level shooting percentages across eras and potentially fit a model that includes season as a feature to estimate how location's effect on success has shifted over time.

RQ3 — Rank Correlation and Visualization

We'll build a shot-quality-adjusted player metric directly from the RQ1 model — something like expected FG% above league average. We'll then compute the traditional metrics (FG%, eFG%, TS%) on the same dataset for an apples-to-apples comparison. Spearman's rho will quantify how similar the two rankings are, and a slope chart will visualize which specific players move up or down the most, under the adjusted metric. The interpretation focuses on whether certain player types or roles systematically get re-ranked.

Computational Methods and Outputs

The specific techniques we expect to use:

- Logistic regression (baseline classifier for shot success)
- Gradient-boosted trees, likely XGBoost or LightGBM (main shot success model)
- Residual analysis (player and team aggregations of model error)
- K-means or hierarchical clustering (player archetype identification for RQ1)
- Kernel density estimation (smoothed shot success heat maps for RQ2)
- Paired statistical tests (longitudinal comparison for RQ2)
- Spearman rank correlation (RQ3)
- Slope charts and rank-shift visualizations (RQ3)

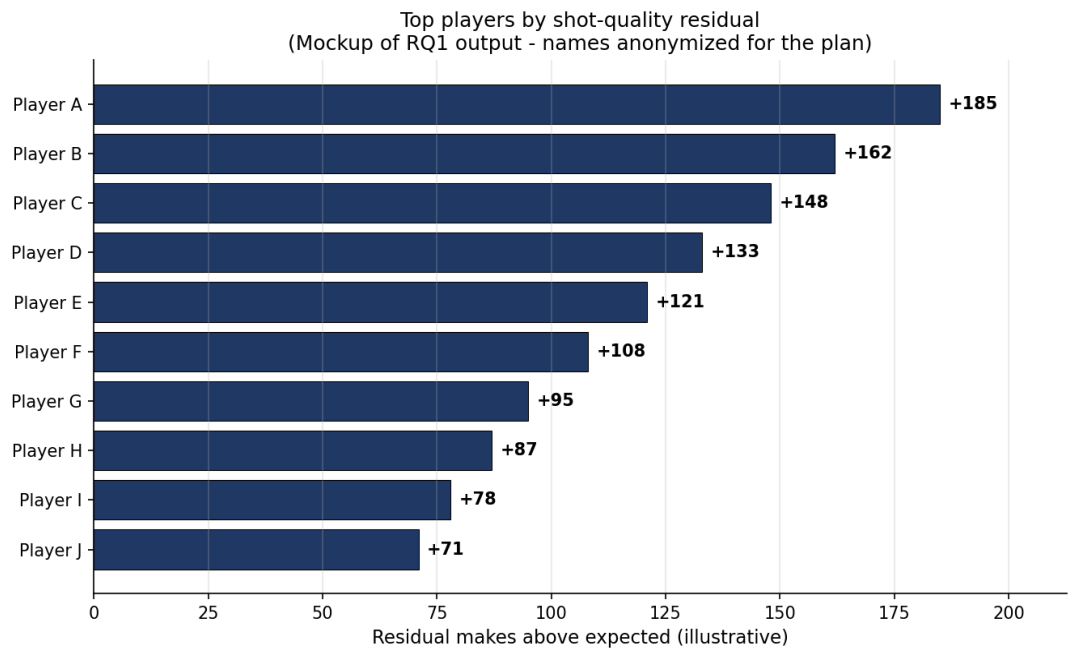
Output Mockups

These are early-stage mockups showing the shape our final outputs will take. The actual outputs will be generated from our pulled data once the pipeline is in place.

RQ1 — Overperformer Residual Leaderboard and Archetypes

Our primary RQ1 output is a leaderboard of the top players (and separately, top teams) by total residual shot value over the tracking era. Each row shows the player's actual makes, expected makes from the model, and the difference. A secondary output describes the player archetypes

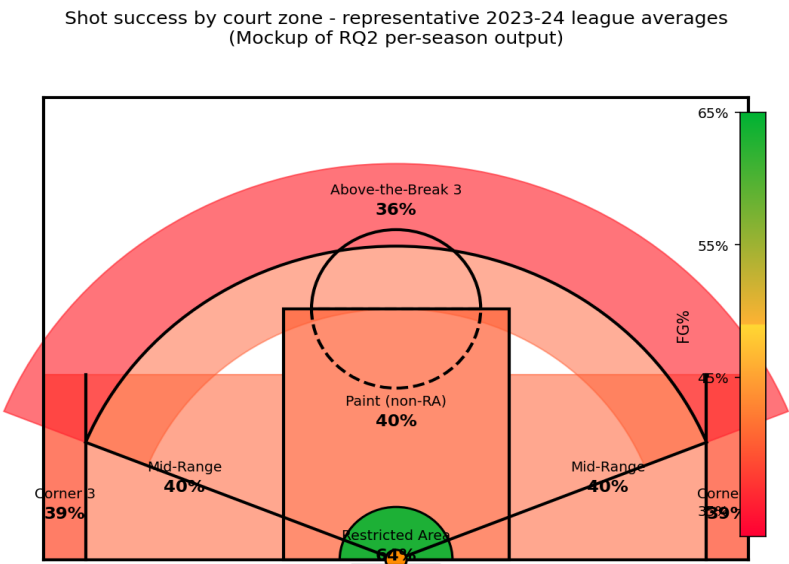
that emerge from clustering — for example, 'elite primary creators,' 'high-volume catch-and-shoot specialists,' and 'efficient interior finishers.'



Mockup of the RQ1 leaderboard. Player names are anonymized for the plan; real results will name actual players from our analysis.

RQ2 — Heat Maps Across the Tracking Era

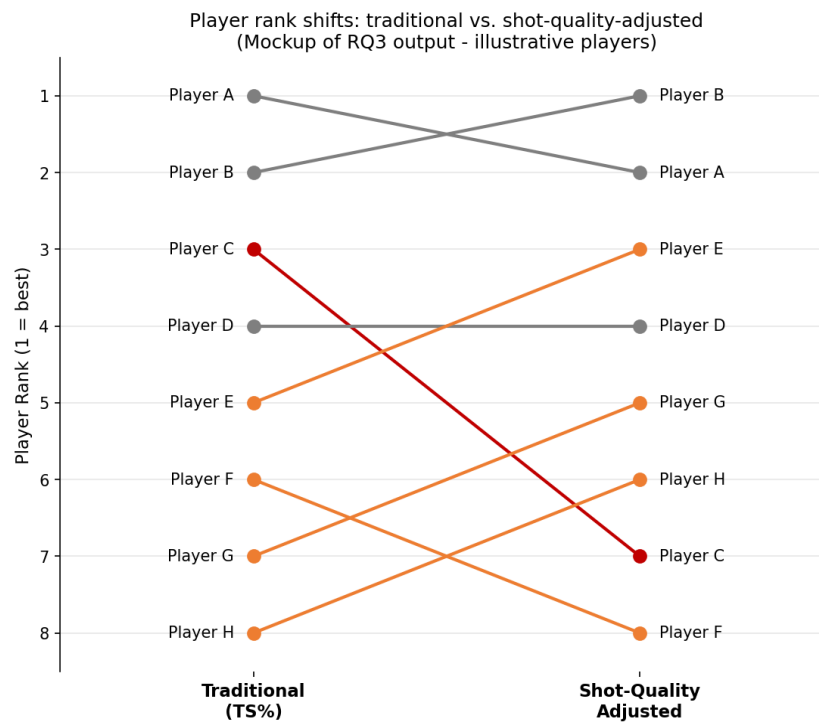
Our RQ2 output is a series of smoothed shot-success heat maps, one per season, that can be viewed side by side to show how the geography of effective shooting has changed. A summary chart shows zone-level shooting percentages plotted over time.



Mockup of the per-season heat map output. Real heat maps will be generated per season from our shot detail data.

RQ3 — Rank-Shift Slope Chart

The RQ3 output is a slope chart with two parallel rank axes — players ranked by traditional efficiency metrics on the left, ranked by our shot-quality-adjusted metric on the right — with lines connecting each player's two ranks. Large slopes flag the players whose evaluation changes most under the adjusted metric. We'll also report the Spearman rank correlation as a single summary statistic.



Mockup of the RQ3 slope chart. Real player names and rankings will replace the placeholders.

Campaign Implementation

Once the analysis is done, the results have clear applications for several groups in and around the NBA.

Team front offices and basketball operations staff are the most direct audience. The shot-quality-adjusted player rankings from RQ3 give general managers a sharper tool for free-agent evaluation, contract negotiation, and trade analysis. The archetype clusters from RQ1 help inform roster construction — knowing which kinds of players consistently exceed their expected value lets teams target specific profiles in the draft and trade market.

The NBA league office is interested in the league-wide trends surfaced by RQ2. The continued evolution of shot selection has implications for rules changes, court dimensions, and how the game is officiated and presented. Quantifying how location's value has shifted gives the league empirical grounding for decisions about the game's strategic direction.

Broadcast partners (Disney/ABC/ESPN, NBCUniversal, Amazon Prime Video) benefit from richer storytelling around individual player performance. A broadcast graphic showing that a player is among the league leaders in shot-quality-adjusted scoring tells a more meaningful story

than raw shooting percentages, especially for role players whose contributions get overlooked in traditional box scores.

Player agents and media analysts can use the same metrics to advocate for their clients or surface narratives traditional stats miss — particularly for low-usage specialists whose value gets undersold by conventional efficiency measures.

Timeline & Milestones

Below is our planned schedule for the rest of the semester. This is an estimate, as the project plan is a living document, and we'll communicate any significant schedule shifts.

Week	Dates	Milestones
Week 2	May 25 – 31	Project Plan submitted; Research Questions submitted; repo and data pipeline scaffolded; Germain begins running the full NBA Stats API pulls; Cole drafts and submits the Literature Review.
Week 3	June 1 – 7	Data pulls complete and enriched shot table built; Calder runs EDA on the shot-level data; Marc prototypes the shot-success model on the 2014-15 Kaggle dataset; Cole drafts the Methodology and EDA writeups, submitted as each is finalized.
Week 4	June 8 – 14	Marc trains the full multi-season model and runs residual analysis + clustering for RQ1; RQ2 heat-map series and RQ3 rank comparison generated; Cole drafts the Ethics and Data Visualizations writeups, submitted as each is finalized.
Week 5	June 15 – 21	Analysis, Challenges, Recommendations & Next Steps, Abstract, and References drafted; Rough Draft compiled and reviewed by the team. Rough Draft due Sunday, June 21 at 11:59 PM.
Week 6	June 22 – 26	Instructor feedback received on the Rough Draft; revisions incorporated within the 50% grade-recovery window; source code Appendix finalized; full document polished and proofread. Final Report due Friday, June 26 at 11:59 PM.

Review of the Literature

Our project sits at an important position between three threads in basketball analytics research: the broader rise of quantitative decision making in the NBA, the modeling of shot quality and expected possession value, and the use of those models to evaluate individual players. This review walks through each of those threads in turn, starting with the analytics era as a whole, narrowing into the theoretical and modeling work that defines shot quality, and ending with the modern player evaluation literature and the ethical considerations that come with it. The goal is to position our own work in a field that has matured rapidly over the last two decades and is still actively producing new methodology today.

The Analytics Era in the NBA

Quantitative analysis of basketball games dates back to the late 1940s (Poropudas & Halme, 2023), but the modern era of basketball analytics is widely understood to have started in the early 2000s with the publication of Dean Oliver's *Basketball on Paper* (Oliver, 2004). Oliver argued for replacing simple per-game averages with tempo-adjusted metrics such as points per possession, and he proposed the “four factors” framework — effective field goal percentage, free throw rate, offensive rebounding rate, and turnover rate — as the four independent dimensions that drive team success. Subsequent work by John Hollinger and David Berri extended this approach to composite measures of individual player value and to the connections between productivity, salaries, and team winning percentage (Poropudas & Halme, 2023). Poropudas and Halme (2023) themselves revisit Oliver's framework in light of the last two decades of league play and show that the relationship between the four factors and team efficiency ratings is non-linear, with the marginal impact of any one factor depending on the values of the others.

The infrastructure underneath these models has changed almost as much as the models themselves. Box-score summaries gave way to possession-by-possession, play-by-play data in the late 1990s, and the NBA installed optical player and ball tracking in every arena starting in the 2009–10 season (Poropudas & Halme, 2023). That tracking data is what makes the modern shot-quality tracking possible, and it is the same data feeding our own analysis. Combined with shifts like the “three-point revolution” — the league-wide move away from mid-range shots toward layups and threes — the analytics era has reshaped how teams build rosters, allocate possessions, and evaluate the players who take their shots.

Shot Selection as an Optimization Problem

One of the earliest theoretical contributions in this space comes from Skinner (2011), who framed shot selection as an optimization problem rather than an efficiency calculation. Skinner argued that the value of any given shot is not just its own expected points, but its expected points relative to

the shot the offense could have generated by passing it up. Under this view, a player who only takes shots with very high expected value is not necessarily an efficient scorer, because they may be passing up enough other reasonable shots that the team's overall scoring rate suffers. Skinner's framing introduced the vocabulary of marginal value, opportunity cost, and expected outcome that the rest of the shot-quality framework builds on, and it is the conceptual model of every “expected” metric we use in our own work.

Modeling Shot Quality and Expected Possession Value

A highly influential body of modeling work in this area comes from Cervone, D'Amour, Bornn, and Goldsberry (2014), whose POINTWISE framework introduced the concept of *expected possession value* (EPV). Rather than evaluating possessions only at the moment a shot is taken, POINTWISE models the value of a possession at every moment — every pass, every dribble, every off-ball cut — as a function of the current spatial configuration of the players and the ball. The accompanying technical paper develops the multiresolution stochastic process model that makes those moment-by-moment estimates numeric (Cervone, D'Amour, Bornn, & Goldsberry, 2014). Together, these two papers established that high-frequency optical tracking data could be used not just to describe what happened on a possession, but to estimate what *should have* happened given the choices available to the offense. That distinction informs the methodological foundation of our shot-success model.

Modern Player Evaluation Through Shot Quality

More recent work has applied the EPV lineage to specific questions about individual player value. A 2024 paper introduces *Expected Points Above Average* (EPAA), a Bayesian hierarchical model that compares a player's scoring contributions against what an average player would have produced from the same shots, giving a per-player measure of shot-quality-adjusted performance (Expected Points Above Average, 2024). The approach is a direct representation of what we propose for RQ1 — taking residuals between actual makes and model-expected makes, then using those residuals to surface players who consistently outperform expectations.

Kono and Fujii (2024) extend player evaluation metrics in a different direction by modeling off-ball scoring opportunities rather than just the shooter's expected outcome. Their BMOS (Ball Movement for Off-ball Scoring) and BIMOS (Ball Intercept and Movement for Off-ball Scoring) models use STATS SportVU tracking data from 630 NBA games in the 2015–16 regular season to estimate the probability that an off-ball player will end up in a position to score, accounting for passing, dribbling, and the chance of interception along the way. The work makes a useful theoretical point that although data-driven EPV models are powerful, theory-based models grounded in physical parameters of the players and the ball can offer more interpretable results, which matters when the goal is evaluating decisions rather than just predicting outcomes.

The methodological foundations of residualized player metrics have also received recent attention. Bajons and Kook (2025), working primarily in soccer, present a general framework for player evaluation metrics built on differences between actual and expected outcomes. They show that the commonly used “above expectation” metrics — Goals Above Expectation in soccer, and by extension our shot-quality residuals in basketball — are mathematically related to Rao's score test in parametric regression models, and they propose residualized versions of those metrics that forms valid inference even when the underlying expected-outcome model is a flexible machine learning estimator. Their framework directly addresses one of the open methodological questions in our own RQ1: how confident can we be that a player's positive residual reflects real skill rather than noise? Their answer is the kind of refinement we may incorporate as we develop our analysis.

Ethics and Fairness in Algorithmic Player Evaluation

The shot-quality and player evaluation models discussed above are not academic exercises. They feed many decisions about contracts, trades, and roster construction, which means the ethical implications of how they are built and used matter as much as their statistical accuracy. The standard reference for thinking about these implications is Barocas, Hardt, and Narayanan's *Fairness and Machine Learning: Limitations and Opportunities* (Barocas, Hardt, & Narayanan, 2023), which lays out the major frameworks for evaluating algorithmic fairness including group-level fairness (parity of outcomes across demographic or role groups), individual fairness (similar individuals receiving similar predictions), and the trade-offs between them. Applied to NBA player evaluation, these frameworks raise concerns that the shot-quality literature has not always engaged with directly. A model that ranks players purely by shot-quality residual may systematically disadvantage players whose roles force them into low-expected-value shots — primary creators carrying heavy usage, or bigs setting screens in physical interior situations — even if those players are performing well within their assigned role. By building a shot-quality model that addresses these issues, we can develop a model that may well be used in NBA analytics themselves.

Where Our Project Fits

Our work builds directly on the EPV and shot-quality models rooted in the Cervone et al. (2014) papers, while drawing on the residualized-evaluation methodology of Bajons and Kook (2025) and the modern Bayesian approach of EPAA (2024). The three research questions we proposed map well onto these threads: RQ1, which identifies players and teams that consistently outperform their expected shot value, is a direct application of residualized player evaluation to NBA shot data; RQ2, which traces how shot success by court location has changed across the tracking era, extends the longitudinal view of the analytics era that Poropudas and Halme (2023) describe at the team level; and RQ3, which compares shot-quality-adjusted rankings against traditional efficiency metrics, connects our work to the broader question of whether the shot-quality lens tells a different

story than the box-score-derived metrics that still dominate NBA discourse. The ethics framing from Barocas et al. (2023) runs through all three questions, particularly in how we interpret which players and roles the model rewards or penalizes.

Final Research Questions

We began the project with three research questions in our project plan, and after developing the methodology and the model we carried all three through the full analysis without dropping or merging any of them. Each one is answered in the Analysis section. The three final research questions are:

- 1. Which players and teams consistently outperform their expected shot value, and what distinguishes them?**
- 2. How has shot success by court location changed across the 2013-present tracking era?**
- 3. Does shot-quality-adjusted player ranking differ noticeably from traditional efficiency metrics (FG%, eFG%, TS%)?**

The only refinement we made over the course of the project was to the data each question draws on rather than to the questions themselves. The shot-quality model behind RQ1 and RQ3 is built on the single season (2014-15) for which per-shot tracking features are publicly available, while RQ2, which is about change over time, uses the full thirteen seasons of location data. The questions we set out to answer are the questions we answered.

Exploratory Data Analysis

Data sources and setup

The data for this project comes from the NBA Stats API (accessed through the open-source `nba_api` Python package) supplemented by two Kaggle datasets. The NBA Stats API gives us shot-level detail for every field goal attempt from the 2013-14 season through the current 2025-26 season — roughly two million shots in total — and includes the matching game logs, player bios, and team metadata. The two Kaggle supplements are the 2014-15 NBA Shot Logs dataset (dansbecker/nba-shot-logs, about 128,000 shots used for early prototyping) and the NBA Stats 1947-present dataset (sumitrodata/nba-aba-baa-stats, used for long-term player history). All sources map to NBA.com identifiers so joins are ID-based rather than name-based.

After the data pull, we combined the thirteen seasons of shot data into a single table with an added SEASON column derived from the file name. The combined raw shot table covers every field goal attempt across the tracking-data era. The dependent variable we care about is SHOT_MADE_FLAG, a binary indicator of whether a given shot went in.

Variables analyzed

The variables we focused on for this exploratory pass are the ones most directly related to whether a shot is likely to go in. They are:

- SHOT_MADE_FLAG — dependent variable, binary (1 if the shot was made, 0 if missed)
- SHOT_DISTANCE — distance from the basket in feet
- LOC_X, LOC_Y — court coordinates of where the shot was taken
- SHOT_ZONE_BASIC — categorical zone label (Restricted Area, In The Paint Non-RA, Mid-Range, Left Corner 3, Right Corner 3, Above the Break 3, Backcourt)
- SHOT_TYPE — either "2PT Field Goal" or "3PT Field Goal"
- PERIOD — quarter of the game
- ACTION_TYPE — descriptive shot type (jump shot, layup, dunk, etc.)
- SEASON — derived from the file name; identifies which year a shot is from

In addition to these raw variables, we derived a few simple summary measures during the analysis itself: per-season averages for shot distance, share of attempts by zone, and field goal percentage by zone. These are not new columns added to the table, they're aggregations we computed for the visualizations that follow.

How shot selection has changed over time

The first thing worth looking at is how the league's shot selection has changed across the tracking-data era, because the answer is dramatic. Appendix A shows the average shot distance per season from 2013-14 through 2025-26. The trend is steadily upward through the late 2010s and then plateaus in the most recent seasons, consistent with a league that gradually moved further from the basket and then settled into a new normal.

Appendix B breaks the same trend out by shot type. The share of attempts that are three-pointers rises sharply over the era, while the share of two-point attempts falls by a near-equal amount. This is the "three-point revolution" that's been written about extensively in the analytics literature, and our data shows it very clearly. Appendix C then plots the field goal percentages for two-point and three-point attempts separately. Interestingly, the FG% for each shot type stays flat over the era, even as the volumes shift so dramatically. The league is not getting better at three-pointers — it's just taking more of them.

Appendix D zooms in further by showing the share of attempts taken in each shot zone over time. The two most visible movements are above-the-break threes (rising sharply) and mid-range shots (falling just as sharply). Over our full tracking era, mid-range attempts dropped from about 27 percent of all attempts in 2013-14 to about 10 percent in 2025-26; a 17-percentage point drop. Above-the-break threes went the opposite direction, rising from about 19 percent to about 31 percent. Restricted area attempts dipped slightly; these shifts are the clearest evidence in the entire EDA that shot location is not a static feature of the data we're modeling, and that any model trained across the full era must be able to handle this distributional change.

Which zones are efficient

Looking at field goal percentage by zone aggregated across the full era gives us the efficiency landscape we're modeling against. The numbers (visualized in Appendix E) are roughly:

Zone	FG% (2013–present aggregate)
Restricted Area	63.6%
In The Paint (Non-RA)	42.1%
Mid-Range	40.5%
Right Corner 3	38.7%
Left Corner 3	38.6%
Above the Break 3	35.2%
Backcourt	2.7%

The Restricted Area is overwhelmingly the most efficient place to shoot, followed by a cluster of two-point and corner-three shots that all sit in the high-30s to low-40s, followed by above-the-break threes in the mid-30s. The Backcourt zone is noise, as it's mostly heaves and buzzer-beaters and doesn't behave like a normal shot.

Appendix F plots this same FG% per zone but broken out by season, and the most interesting result is that the relative ranking of zones stays almost perfectly stable across the entire 12-year window. Within zones, though, there are real changes; most notably, Restricted Area FG% rises from about 61% in 2013-14 to about 67% in 2025-26 (a 6-percentage point increase), and In-the-Paint Non-RA also climbs by about five percentage points. Corner threes and above-the-break threes are flat in FG% over the era. The takeaway is that players are not getting better at the three-point shot, they are getting better at finishing at the rim — even as they take fewer shots from inside the arc.

How attempts and FG% changed from 2013-14 to 2025-26

To make the era shift concrete, we computed a direct change table comparing the first season of the tracking era to the most recent season. The result:

Zone	Attempt % 2013-14	Attempt % 2025-26	Δ	FG% 2013-14	FG% 2025-26	Δ
Above the Break 3	19.2%	30.7%	+11.5	35.3%	34.9%	-0.3
Mid-Range	26.8%	10.1%	-16.7	39.5%	41.6%	+2.1
Restricted Area	32.3%	28.4%	-3.9	60.9%	67.0%	+6.1
In The Paint (Non-RA)	14.9%	20.0%	+5.1	39.3%	44.4%	+5.0
Right Corner 3	3.1%	5.2%	+2.0	38.8%	38.4%	-0.4
Left Corner 3	3.5%	5.6%	+2.0	39.2%	38.7%	-0.5

The pattern is clear: the league moved its attempt volume from the mid-range to above-the-break and corner threes, and the underlying efficiency of those threes did not improve, while the efficiency of the shots taken at the rim improved.

Appendix G shows the same era shift visually — a side-by-side scatter plot of 10,000 sampled shot locations from 2013-14 alongside 10,000 from 2025-26. The 2026 plot has a much denser concentration of dots beyond the three-point arc, and a visibly thinner mid-range band compared to 2013.

Winning teams versus losing teams

We also looked at how the shot profiles of winning teams differ from losing teams, joining each shot to its win/loss outcome. The aggregated 2013-2026 picture is that winning teams shoot much better in every single zone, with the gaps clustered around four to six percentage points. The largest gaps appear in the three-point zones (above-the-break threes are 5.85 points higher for winning teams; corner threes are 6.25 and 6.46 points higher), and the smallest gap is in the restricted area at 4.30 points.

A more interesting follow-up is whether that winning-team advantage has changed over time. Comparing 2013-14 to 2025-26 directly, the win/loss FG% gap has narrowed in most zones — for example, the above-the-break three gap shrank from 6.21 in 2013-14 to 4.96 in 2025-26, and the mid-range gap shrank from 3.98 to 2.65. This is consistent with the broader idea that league-wide shooting talent has converged: as more teams have built around modern shooting, the difference between winning and losing teams from any specific spot has gotten smaller, even as the value of the attempts themselves has shifted dramatically. This is an interesting finding because it suggests that the gap between good and bad teams in our era is less about shot success rate within zones and more about which zones each team chooses to take their shots from.

Implications for the model

Several findings from this EDA shape how we're approaching modeling work.

First, shot location is the single most informative feature for shot success — which the EDA confirms and which Marc's initial logistic regression baseline already corroborated (his strongest

feature by coefficient magnitude was SHOT_DISTANCE, negative; his strongest positive was CLOSE_DEFENDER_DISTANCE). Any expected-shot-value model has to capture spatial features explicitly.

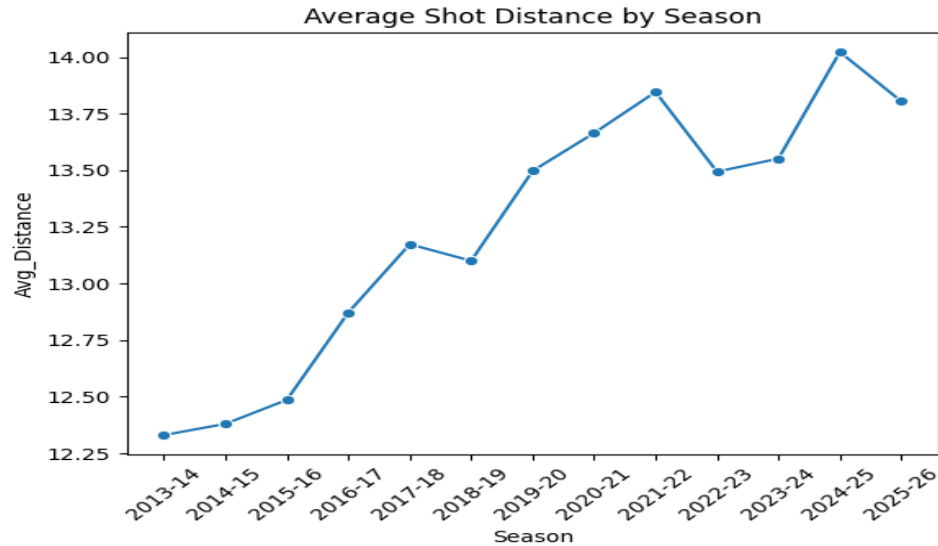
Second, the league-wide attempt distribution has shifted dramatically over the tracking era, but the within-zone efficiency rankings have stayed stable. This means a model trained on the full 2013-present window can use zone-level features without worrying that the *value* of being in a given zone has changed, even though the *frequency* of being in each zone has shifted.

Third, the winning-team advantage in shot success is smaller and more uniform than we expected, which strengthens the case for our RQ1 approach of using residual analysis to identify players and teams that exceed model expectations. If winning teams were simply better at making any given shot, residuals would be noisy; the fact that the gap is moderate and varies by zone means residual-based player evaluation can carry good signal.

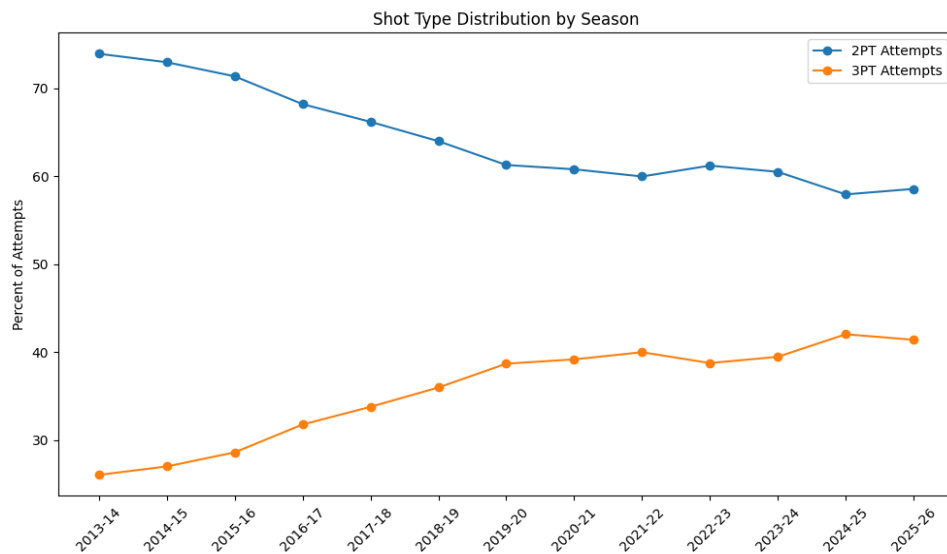
Finally, this EDA confirms RQ2 has substantive findings to surface: the geography of shot success has clearly evolved across the tracking era, and that evolution is rich enough to support good analysis rather than a flat trend.

Appendix

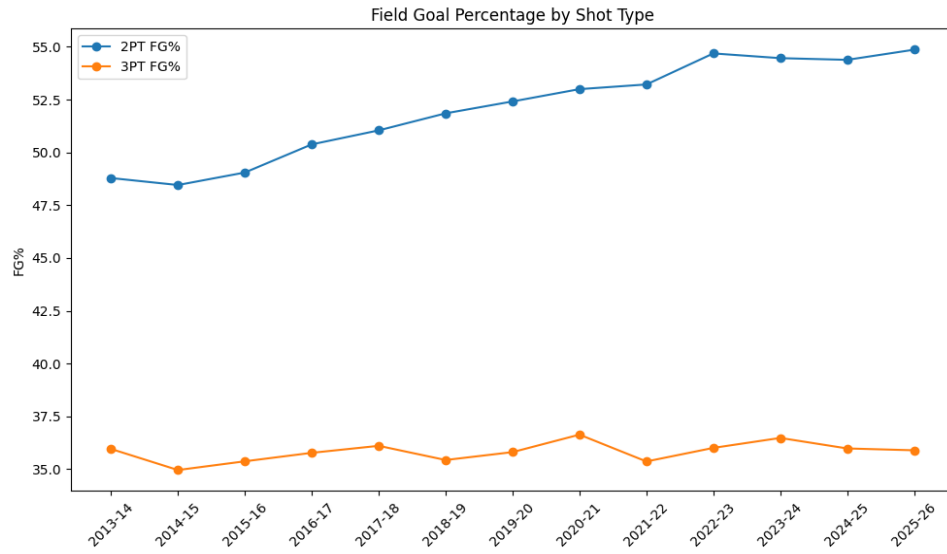
The figures below are the visualizations referenced in the prose above. Source code (including the queries and data transformations behind each chart) will be included in the Appendix of the Final Report.



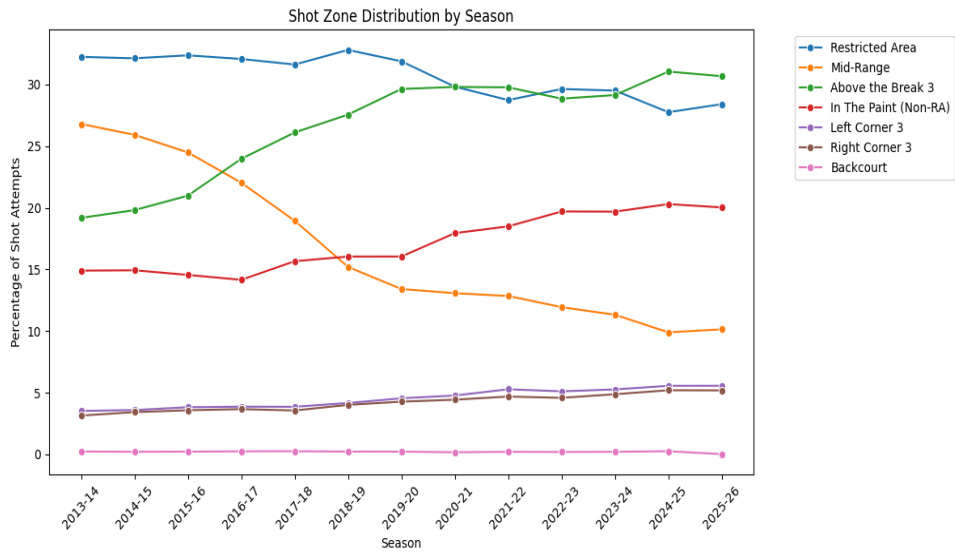
Appendix A: Average shot distance by season, 2013-14 through 2025-26.



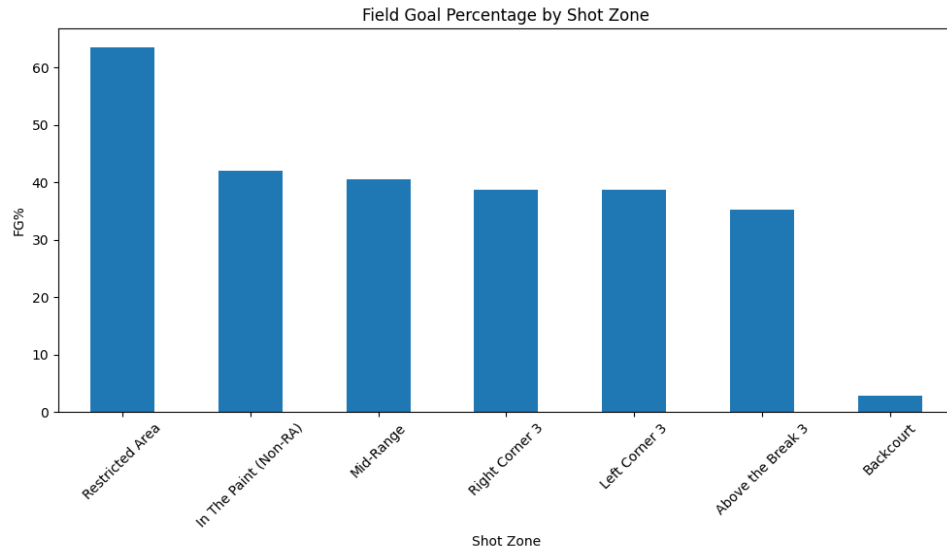
Appendix B: Share of attempts by shot type (2PT vs 3PT) over time.



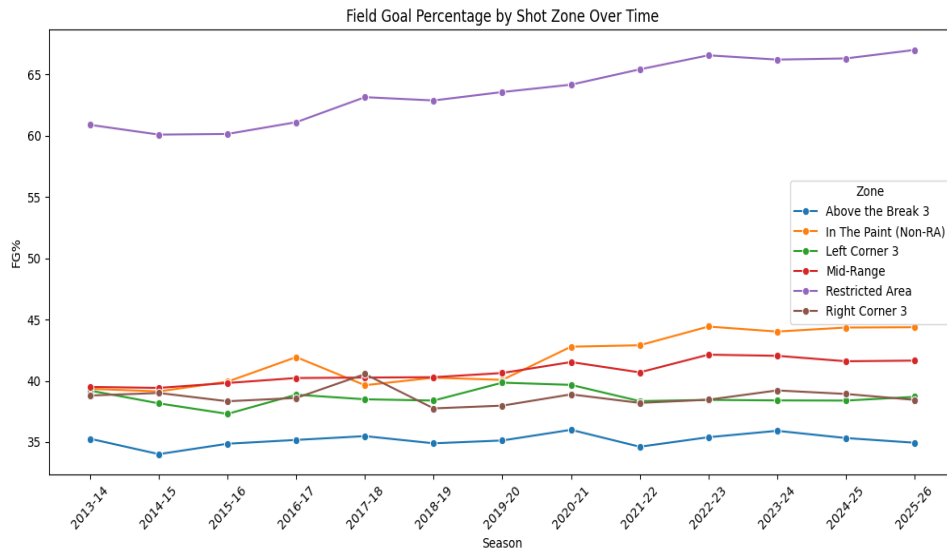
Appendix C: Field goal percentage by shot type (2PT vs 3PT) over time.



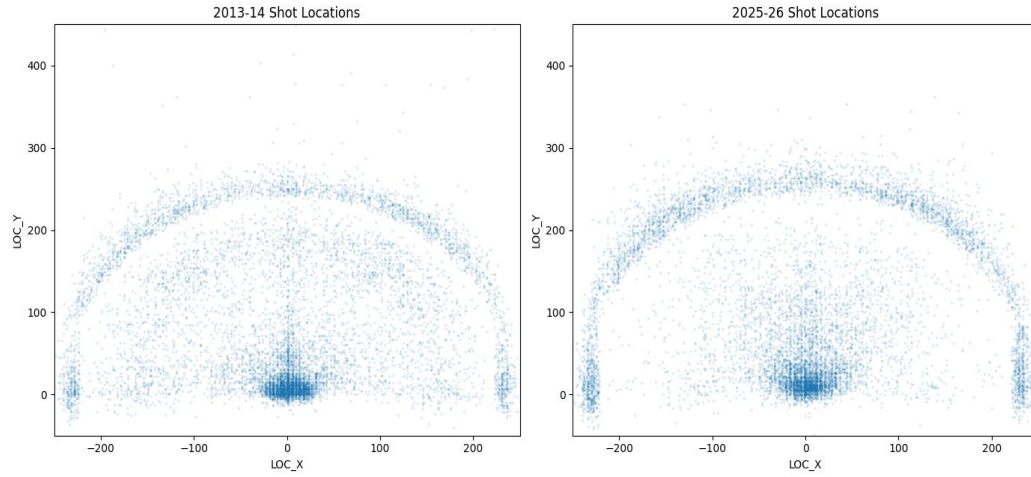
Appendix D: Share of attempts by shot zone over time.



Appendix E: Field goal percentage by shot zone (aggregate 2013-present).



Appendix F: Field goal percentage by shot zone over time.



Appendix G: Shot locations 2013-14 vs 2025-26, side-by-side scatter of 10,000 sampled shots from each season.

Methodology

This section describes the analytical procedures we're using to answer each of our three research questions. Following the project plan, the work is organized by research question, with each RQ getting a description of the modeling approach, the specific techniques being applied, and how we evaluate the results. As the instructor noted in the project plan check-in, the methodology section is a forecast at this stage — we're committing to a set of techniques and an evaluation strategy, and as the analysis matures we may refine specifics like model choice or feature set.

RQ1 — Which players and teams consistently outperform their expected shot value, and what distinguishes them?

RQ1 is a layered analysis rather than a single method, because the question itself has two parts: identifying overperformers, and explaining what makes them different. To answer the first part we build an expected-shot-value model; to answer the second we aggregate that model's errors and group the resulting overperformers using unsupervised techniques.

Data preparation. The base data for this question is the enriched shot table assembled by our cleaning pipeline, with one row per field goal attempt across the 2013-14 through 2025-26 tracking era. We use shot context features that the EDA confirmed are the most informative: shot distance, court coordinates, shot zone, shot clock, dribbles before the shot, touch time, defender distance, shot type, and game-state variables like period and final margin. Features that would leak the outcome (FGM, PTS, SHOT_RESULT) are excluded explicitly. The dependent variable is SHOT_MADE_FLAG, a binary indicator of whether the shot went in.

Evaluation. Because the dataset is large (roughly two million shots), a straightforward 80/20 stratified train/test split provides enough holdout data for stable performance estimates. For the final modeling pass we'll also use 5-fold cross-validation to confirm that performance is not dependent on a single split. The primary evaluation metrics are accuracy and ROC-AUC (the area under the receiver operating characteristic curve), which together measure both how often the model is right and how well it separates makes from misses across thresholds.

The techniques we're using for RQ1:

- **Logistic regression (baseline classifier).** Logistic regression predicts the probability of a binary outcome by fitting a linear combination of features through a logistic link function. It's interpretable — each feature gets a signed coefficient that tells us whether it increases or decreases the predicted probability of a made shot — and it gives us a baseline that more complex models have to clearly outperform to earn their additional complexity. Our prototype baseline on the Kaggle 2014-15 shot logs returned an accuracy of approximately 0.61 and a ROC-AUC of approximately 0.63, with SHOT_DISTANCE showing the strongest negative coefficient (farther shots are harder) and CLOSE_DEF_DIST showing the strongest positive coefficient (more defender space helps). These directions match the basketball intuition the EDA also surfaced, which gives us confidence that the model is learning real signal.
- **Gradient-boosted trees (XGBoost or LightGBM).** Logistic regression assumes feature effects are linear and additive, which limits the relationships it can capture. Shot success is not linear: the value of an extra foot of defender space depends on the shot distance, which depends on the shot clock, and so on. Gradient-boosted trees build a sequence of

small decision trees where each new tree corrects the errors of the previous ones. The ensemble captures non-linear interactions between features without us having to specify them in advance. We expect this model to outperform the logistic baseline on both accuracy and ROC-AUC, and we'll quantify the improvement so it's visible whether the added complexity is worth it.

- **Residual aggregation (per player and per team).** Once we have the trained shot-success model, we score every shot in the dataset to produce a model-expected outcome. We then compute residuals as actual makes minus model-expected makes, aggregated per player and per team. Players with consistently positive residuals are our overperformers: they make more shots than the model expected given the context of each shot they took. This is the methodological core of the EPV / shot-quality literature reviewed earlier — separating shot selection from shot-making skill.
- **K-means clustering (player archetypes).** Identifying who is overperforming is only half the question; the second half is what distinguishes them. To answer that, we cluster overperformers using playing-style features such as usage rate, shot mix by zone, position, team pace, and average shot distance. K-means partitions the players into a small number of groups by minimizing the variance within each group, and we examine the resulting clusters to give them descriptive archetype labels (for example, "primary shot creators," "low-usage corner-three specialists," "interior finishers"). Hierarchical clustering may be used as a confirmation pass to check whether the K-means partitions are robust.

RQ2 — How has shot success by court location changed across the 2013-present tracking era?

RQ2 is a descriptive longitudinal analysis rather than a predictive modeling problem. The goal is to characterize the geography of shot success in each season of the tracking era and quantify how it has changed, rather than to predict whether a specific shot will go in.

Data preparation. We use the same enriched shot table as RQ1, but here the analysis works at the season level rather than the shot level. For each of the thirteen seasons, we group shots by SHOT_ZONE_BASIC (the NBA's standard zones: restricted area, in-the-paint non-RA, mid-range, left corner 3, right corner 3, above the break 3, backcourt) and compute the field goal percentage within each zone. The EDA already produced this view of the data, which establishes both the underlying pattern and the longitudinal trend we're modeling here.

Evaluation. Because RQ2 is descriptive rather than predictive, the "evaluation" question is whether the differences we observe between seasons are statistically meaningful or could be attributed to random variation. Each season includes roughly 200,000 shots, which gives us strong statistical power to detect even modest changes, but we still want to apply formal tests.

The techniques we're using for RQ2:

- **Spatial binning by NBA zone.** Rather than treating the court as a continuous (x, y) surface, we use the league's standard categorical zones. This makes the results directly comparable to published basketball analytics, gives every interpretation a familiar vocabulary, and matches how teams and broadcasters actually discuss shot location.

- **Kernel density estimation (smoothed heat maps).** As a complement to the zone view, we use kernel density estimation to produce smooth heat maps of shot success across the entire half-court. KDE turns thousands of discrete shot points into a smooth probability surface by spreading a small kernel around each shot location. We generate one heat map per season, which then can be compared visually side-by-side to show how the geography of effective shooting has shifted.
- **Paired statistical tests across seasons.** To quantify whether zone-level FG% differences across the era are real, we use paired comparisons of zone shooting percentages across paired seasons. This captures whether the per-zone change is larger than what we would expect from natural variation alone.
- **Generalized linear model with season as a feature.** As a complementary approach, we fit a model of shot success that includes SEASON as a feature alongside shot zone and other context. The coefficient on the interaction between season and zone tells us how the effect of location on shot success has shifted across the tracking era. This provides a single, condensed numeric summary of the longitudinal trend that supports the descriptive heat-map visualizations.

RQ3 — Does shot-quality-adjusted player ranking differ noticeably from traditional efficiency metrics (FG%, eFG%, TS%)?

RQ3 is a comparison of two rankings — one based on traditional efficiency metrics, one based on a shot-quality-adjusted metric we derive from the RQ1 model — and a test of whether they tell meaningfully different stories about which players are most efficient.

Data preparation. From the RQ1 model, we construct a shot-quality-adjusted player metric. The most natural construction is expected FG% above league average: for each player, we sum their model-expected makes and compare to their actual makes, normalized by attempts. We then compute the three traditional efficiency metrics on the same player population:

- **Field Goal Percentage (FG%).** Made shots divided by attempted shots; the most basic measure.
- **Effective Field Goal Percentage (eFG%).** FG% adjusted to give threes proper credit; $(FGM + 0.5 \times 3PM) / FGA$.
- **True Shooting Percentage (TS%).** The most complete traditional measure, includes free throws and properly weights threes: $PTS / (2 \times (FGA + 0.44 \times FTA))$.

These three are well-established in the public basketball-analytics vocabulary and represent the metrics we're testing our shot-quality-adjusted measure against.

Evaluation. RQ3 is a ranking comparison, so evaluation is about whether the orderings are similar or different.

The techniques we're using for RQ3:

- **Spearman rank correlation.** Spearman's rho measures how similar two rankings are on a scale from -1 to +1. A value near +1 means the two rankings are nearly identical, a value near 0 means they're effectively unrelated, and a negative value would mean the rankings are inverted. Computing Spearman's rho between our adjusted metric and each

of FG%, eFG%, and TS% gives us a direct quantitative answer to the question of whether the rankings differ.

- **Slope chart / rank-shift visualization.** A scalar correlation tells us how similar the rankings are on average, but it doesn't tell us which players move the most. The slope chart plots two parallel rank axes — players ranked by a traditional metric on the left, players ranked by the adjusted metric on the right — with lines connecting each player's two ranks. Players with the largest slopes are the ones whose evaluation changes most under the shot-quality lens, and those are the cases worth interpreting.
- **Subgroup analysis by player role.** Beyond the headline correlation and visualization, we look at whether certain player types — for example, primary creators, low-usage role players, interior bigs — are systematically re-ranked. This is what gives the comparison interpretive weight rather than just a numeric summary.

Computational methods summary

The complete list of statistical and computational techniques used across all three research questions:

- Logistic regression
- Gradient-boosted trees (XGBoost or LightGBM)
- Residual analysis (player and team aggregations of model error)
- K-means clustering and hierarchical clustering (player archetype identification)
- Kernel density estimation (smoothed shot success heat maps)
- Paired statistical tests (longitudinal comparison of zone-level shooting percentages)
- Generalized linear modeling with categorical interactions (season-by-zone effect)
- Spearman rank correlation (cross-ranking comparison)
- Slope-chart and rank-shift visualizations

All source code implementing these techniques — including model training pipelines, residual aggregation, clustering, heat-map generation, and the rank comparison utilities — will be included in the Appendix of the Final Report rather than in the body, per the assignment specification.

Analysis

This section walks through the results for each research question, the decisions we made at each step, and the reasoning behind them. The work rests on a single shot-success model: For every field goal attempt, the model estimates the probability the shot goes in based on its context, and the gap between what a player or team actually made and what the model expected is what drives the rest of the analysis. We built that model first, confirmed it was learning correctly, and then used it to answer each question in turn.

Building and validating the shot-success model

Our first step was to train a model that predicts whether a single shot is made, using the 2014-15 Kaggle shot logs, since that is the only season for which per-shot tracking features (defender distance, shot clock, touch time, and dribbles) are publicly available. We started with logistic regression as a baseline because it's interpretable and gives us a floor that any more complex model has to clearly beat to be worth the added complexity. The baseline returned an accuracy of about 0.613 and a ROC-AUC of about 0.635 on a held-out test set. More importantly, the coefficients made basketball sense: Shot distance had the strongest negative effect on make probability, and defender distance had the strongest positive effect. Farther shots are harder, and more space helps; exactly what we would expect, which gave us confidence the model was learning correctly rather than noisy.

We then moved to a gradient-boosted tree model (XGBoost), because shot success is not a simple linear function of its inputs: The value of an extra foot of space depends on the distance, which depends on the shot clock, and so on. The boosted model captures those interactions without us specifying them in advance. It improved on the baseline on both measures, reaching about 0.619 accuracy and a 0.644 ROC-AUC. Both models comfortably clear the no-information baseline of 54.8 percent (the accuracy of always guessing the more common outcome, a miss), confirming that shot context carries predictive power. Because the boosted model was the stronger of the two, we used it to generate the expected-make probabilities that feed every result below.

One decision worth noting: The shot-clock field had missing values in the Kaggle data. Rather than drop those rows, which would have pushed the sample below the project's 100,000-observation floor, we imputed the missing values. This kept the full sample of roughly 128,000 shots intact.

Several of our modeling decisions were really choices about what not to do, and they are worth making explicit. We deliberately used the 2014-15 Kaggle shot logs rather than the much larger enriched multi-season table, because the enriched table lacks the shot-difficulty features the model needs; a bigger but less informative dataset would have produced a weaker model. We chose to impute the missing shot-clock values rather than discard those rows, because the loss in sample size was not worth the marginally cleaner data. And we kept the feature set deliberately simple — a handful of interpretable derived features such as shot-distance and touch-time buckets and a late-shot-clock flag — rather than pushing into heavier feature engineering. A model whose inputs we can explain is more useful for a question that is ultimately about understanding player value than a marginally more accurate but opaque model would be.

RQ1 — Which players and teams consistently outperform their expected shot value, and what distinguishes them?

With the model trained, we scored every shot and computed residuals — actual makes minus model-expected makes — aggregated to the player and team level. A consistently positive residual means a player made more shots than the model expected, given the difficulty of the shots they took: The cleanest way we have to separate shot-making skill from shot selection.

It's worth being explicit about why this individual player analysis uses only the 2014-15 season. Measuring shot quality requires knowing how hard each individual shot was, which depends on the per-shot tracking features: Defender distance, shot clock, touch time, and dribbles. Those features are only publicly available for 2014-15; for every other season, the NBA releases shot location and outcome, but not the per-shot tracking detail. Without that detail, the model cannot estimate how makeable a given shot was, so it cannot tell whether a player beat expectations. We therefore evaluate individual players on 2014-15, the one season where the data lets us answer the question properly, and we treat the player and team figures below as specific to that season rather than as timeless judgments. (RQ2, which is about shot location over time rather than individual evaluation, uses all thirteen seasons because location and outcome are available for every year.)

The top individual overperformers in 2014-15 were a mix of elite shooters and skilled scorers:

Player	Makes above expected	Per-shot edge
Chris Paul	+51.0	+5.8%
Kyle Korver	+45.6	+9.5%
Nikola Vučević	+39.7	+4.4%
Stephen Curry	+38.5	+4.0%
Dirk Nowitzki	+34.1	+4.2%

Kyle Korver stands out: on nearly 500 shots he converted at a rate about 9.5 percentage points higher than the model expected, which is an enormous edge over that volume. At the team level, the Los Angeles Clippers led at roughly 78 makes above expectation, well ahead of Toronto (+57) and Minnesota (+49), consistent with a roster built around efficient shot creation and finishing.

Identifying who overperforms is only half the question. To answer what distinguishes them, we clustered the 129 overperforming players using K-means on their shot-selection features: Average shot distance, defender distance, touch time, dribbles, and three-point rate. The clustering produced four clean and interpretable archetypes:

Archetype	Players	Example players
On-Ball Creators	37	Chris Paul, Stephen Curry, Kyrie Irving
Catch-and-Shoot Specialists	36	Kyle Korver, Klay Thompson, Wesley Matthews
Interior Finishers	29	Anthony Davis, DeAndre Jordan, Nikola Vučević

Archetype	Players	Example players
Mid-Range Scorers	27	Dirk Nowitzki, Chris Bosh, Carmelo Anthony

We chose clustering on shot-selection features rather than relying on listed positions because the question is about how a player creates their value, not their nominal position, and the features capture that directly: High dribbles and touch time mark on-ball creators, low touch time and a high three-point rate mark catch-and-shoot specialists, and short average distance marks interior finishers. The archetypes line up with what anyone who follows the league would expect, which is a good sign the grouping is meaningful. The takeaway for RQ1 is that overperformance is not concentrated in one type of player; it shows up across distinct roles, and the model rewards each for doing their specific job well.

RQ2 — How has shot success by court location changed across the 2013-present tracking era?

RQ2 is a descriptive question about change over time, so here we used the full thirteen seasons of location data from the NBA Stats API rather than the single-season modeling sample. We made that decision deliberately: The tracking features that power the model only exist for 2014-15, but shot location and outcome are available for every season, and the question is fundamentally about the geography of shooting across the era.

The headline result is a meaningful shift in where teams shoot. Comparing the first and last seasons of the era by zone:

Zone	Attempt share 2013-14	Attempt share 2025-26	Change
Mid-Range	26.8%	10.1%	−16.7
Above the Break 3	19.2%	30.7%	+11.5
In the Paint (non-RA)	14.9%	20.0%	+5.1
Restricted Area	32.3%	28.4%	−3.9
Corner 3 (combined)	6.7%	10.7%	+4.0

The mid-range collapsed by nearly 17 percentage points of total shot share, by far the largest move, while above-the-break threes rose by about 11 points. This is the well-documented "three-point revolution," and our data shows it cleanly.

What is more interesting is that field goal percentage by zone changed far less than attempt share did. The relative ranking of zones by efficiency stayed about stable across the entire window; the restricted area was always the most efficient zone, the corners always reliable, above-the-break threes always the lowest-percentage shot. One subtle pattern is worth drawing out here: Even as three-point attempts surged, three-point shooting percentage ticked down slightly in every three-point zone, by roughly half a percentage point. That decline is best explained not by players getting worse at shooting, but by the rise in volume itself. As more of the offense moved behind the arc, threes were taken by a wider range of players and from more difficult situations, which pulled the league-wide average down even though the shot itself did not become harder to make.

The one zone that improved is the restricted area, where shooting rose by about 6 percentage points (from roughly 61 percent to 67 percent), meaning players are finishing at the rim more effectively than they did a decade ago. The conclusion for RQ2 is that the era's transformation was about shot selection, not a league-wide improvement in shooting: Teams moved their attempts toward the most efficient zones rather than getting better everywhere at once.

RQ3 — Does shot-quality-adjusted player ranking differ noticeably from traditional efficiency metrics?

For RQ3 we built a shot-quality-adjusted metric from the model — each player's actual field goal percentage minus their model-expected field goal percentage — and compared the ranking it produces against rankings by traditional field goal percentage and effective field goal percentage. To measure how different the rankings are, we used Spearman rank correlation, which is the standard way to quantify whether two orderings of the same set agree.

The correlations were moderate, not high:

Comparison	Spearman's ρ
Shot-quality-adjusted vs FG%	0.387
Shot-quality-adjusted vs eFG%	0.592

A correlation of 0.387 against raw field goal percentage is well below 1.0, which tells us directly that the adjusted ranking tells a meaningfully different story than the metric most casual fans would reach for. The correlation against eFG% is higher (0.592) because eFG% already credits three-pointers, partially capturing shot difficulty, but it still leaves a large gap.

The most revealing view is which players move the most between the two rankings. The pattern sorts cleanly by player type. The biggest risers are perimeter shooters whose raw percentages look ordinary but who consistently make contested shots the model did not expect to fall: Pablo Prigioni climbs about 195 spots, with Mo Williams (+177), Jose Calderon (+176), and Channing Frye (+174) close behind. The biggest fallers are interior players whose high raw percentages come from easy shots near the rim: Omer Asik drops about 233 spots, with Andre Drummond (–210) and Tony Allen (–199) nearby. This is the metric working exactly as designed: it rewards making hard shots and gives little credit for making easy ones. The answer to RQ3 is yes, the adjusted ranking differs substantially from traditional efficiency, and the difference is systematic rather than random; it consistently re-values players according to the difficulty of the shots they take.

Summary of findings

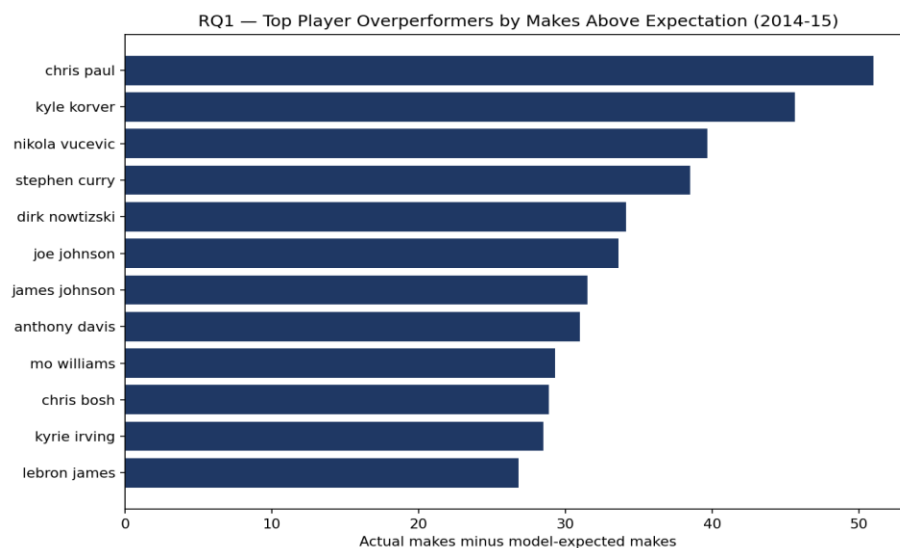
Taken together, the analysis shows that shot context is meaningfully predictive of shot success, that the players and teams who beat the model span distinct and identifiable playing-style archetypes, that the league's decade-long transformation was driven by shot selection rather than league-wide shooting improvement, and that a shot-quality-adjusted view of efficiency reorders players in a meaningful, type-driven way that traditional metrics miss.

Data Visualizations

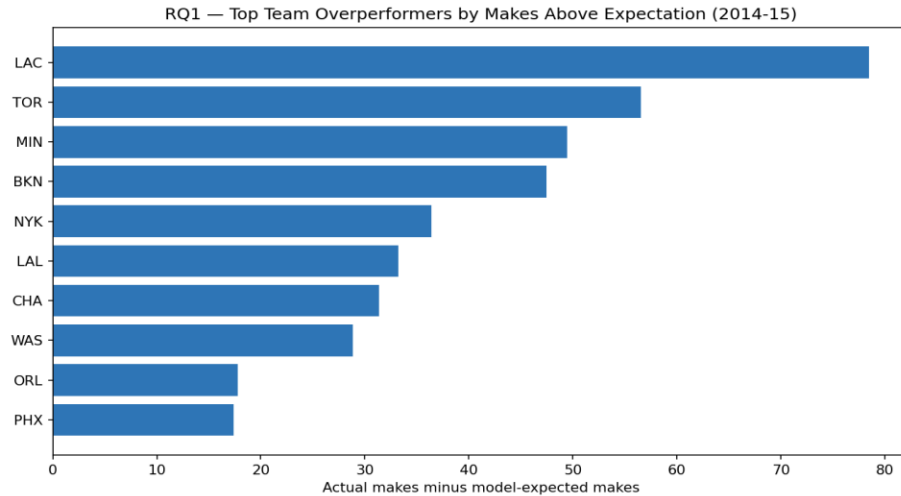
The visualizations in this section are built from the output of our modeling work, and they are meant to support the conclusions we draw for each research question rather than to explore the raw data the way our EDA did. Each chart is paired with a short explanation of what it shows and what it tells us. The figures are organized by research question, followed by a comparison of the two models we trained.

RQ1 — Which players and teams consistently outperform their expected shot value?

Our expected-shot-value model assigns every shot a probability of going in based on its context, which lets us compare how many shots a player actually made to how many the model expected them to make. The difference, summed across a player's season, is their makes above expectation. These player and team results are computed on the 2014-15 season specifically, because measuring shot quality requires the per-shot tracking features — defender distance, shot clock, touch time, and dribbles — that are only publicly available for that season; the analysis section discusses this constraint in more detail.



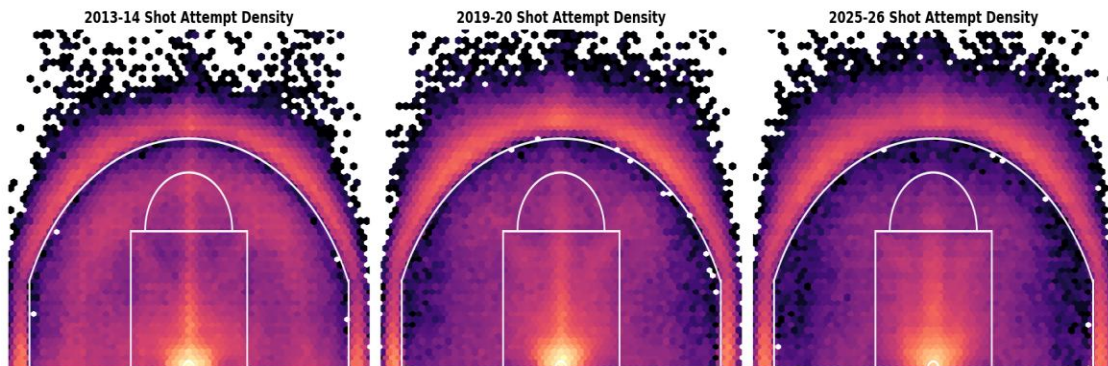
The top player overperformers in 2014-15 are a mix of elite shooters and skilled interior scorers. Chris Paul leads the group at roughly 51 makes above what the model expected, followed by Kyle Korver and Nikola Vučević. What stands out is that several of these players combine high volume with a meaningful per-shot edge — Korver, for example, made his shots at a rate about 9.5 percentage points higher than the model expected, which is an enormous gap for a player taking nearly 500 shots. These are the players the model identifies as beating the difficulty of the shots they take, rather than just taking easy shots.



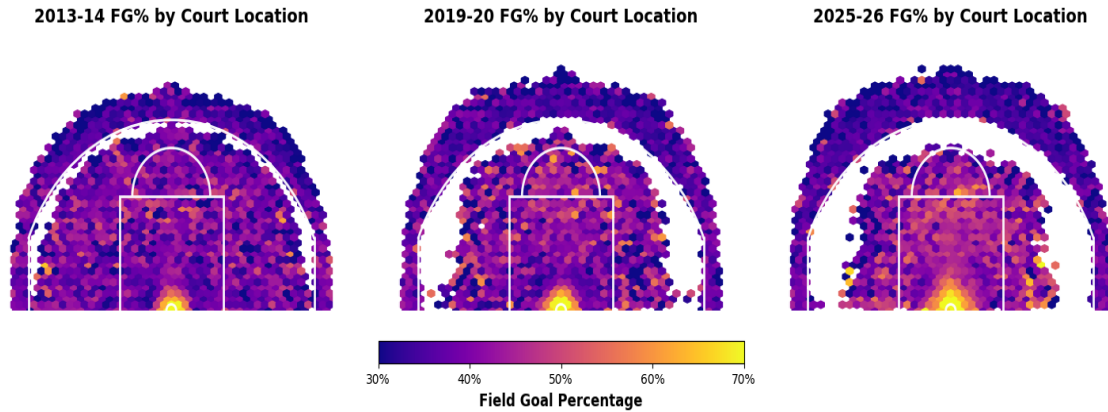
The same logic applies at the team level. The Los Angeles Clippers were the clear top overperforming team in 2014-15 at roughly 78 makes above expectation, well ahead of Toronto and Minnesota. This lines up with what we know about that Clippers roster, which paired elite shot creation with efficient finishing. Aggregating residuals to the team level smooths out the noise from any single player and shows which teams, as a whole, converted their shots better than the model predicted.

RQ2 — How has shot success by court location changed across the tracking era?

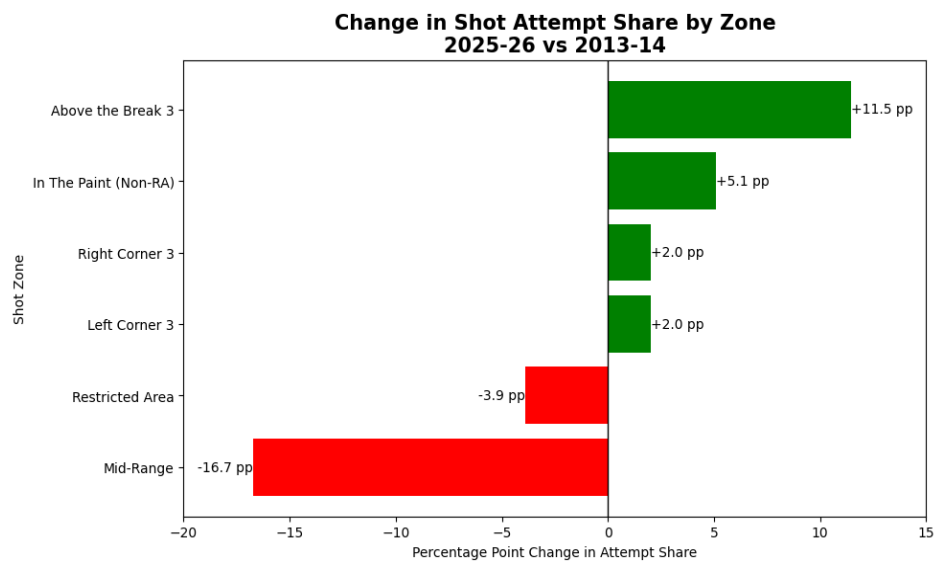
These visualizations trace the geography of shooting across the tracking era. They use the full thirteen seasons of location data rather than the single-season modeling sample, because the question is designed this way.



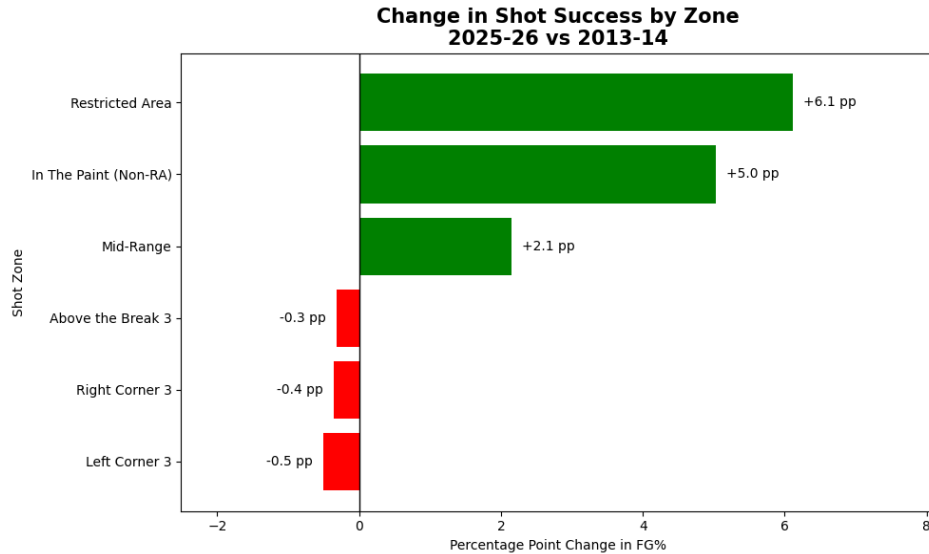
The shot-density maps make the three-point revolution impossible to miss. Moving from 2013-14 to 2019-20 to 2025-26, the dense cluster of attempts steadily migrates outward: the area just inside the arc thins out dramatically, while the volume beyond the three-point line and right at the rim intensifies. The mid-range, which is heavily populated in the 2013-14 panel, is visibly hollowed out by 2025-26.



Coloring the same court by field goal percentage instead of volume tells a complementary story. The restricted area stays consistently the hottest zone across all three seasons, and the corners remain reliably efficient three-point real estate. What is striking is how stable the efficiency surface is even as the volume map changes so much underneath it — teams moved their shots toward the most efficient zones rather than getting dramatically better at any one spot.



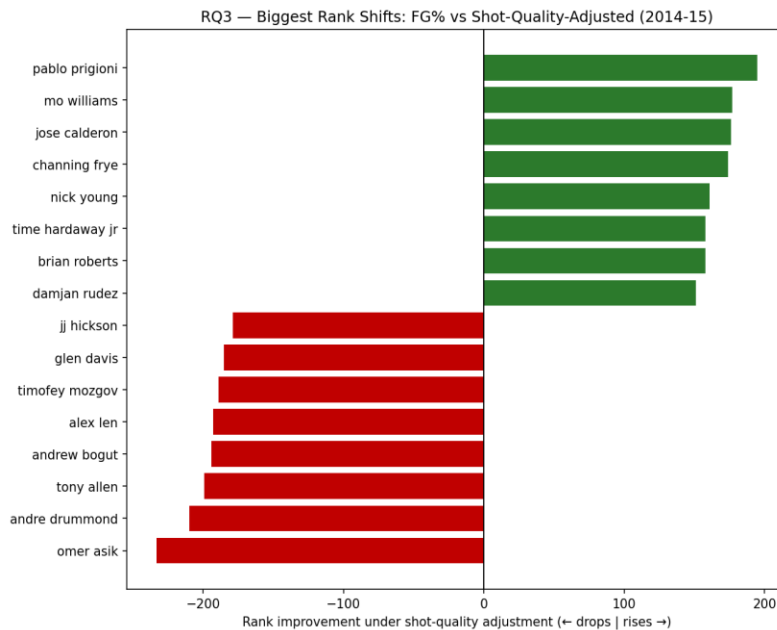
Quantifying the migration by zone confirms the visual story. Between 2013-14 and 2025-26, mid-range attempts fell by roughly 17 percentage points of total shot share — by far the largest move on the chart — while above-the-break threes rose by about 11 percentage points, and the corners and the area inside the arc picked up the rest. This is the clearest single summary of how shot selection has been rebuilt over the era.



Field goal percentage by zone changed far less than attempt share did, which reinforces the point that the era's shift was about where teams shoot rather than how well. The one notable exception is the restricted area, where shooting improved by about six percentage points — players are finishing at the rim more effectively than they did a decade ago, even as the rest of the efficiency landscape held roughly steady.

RQ3 — Does shot-quality-adjusted ranking differ from traditional efficiency metrics?

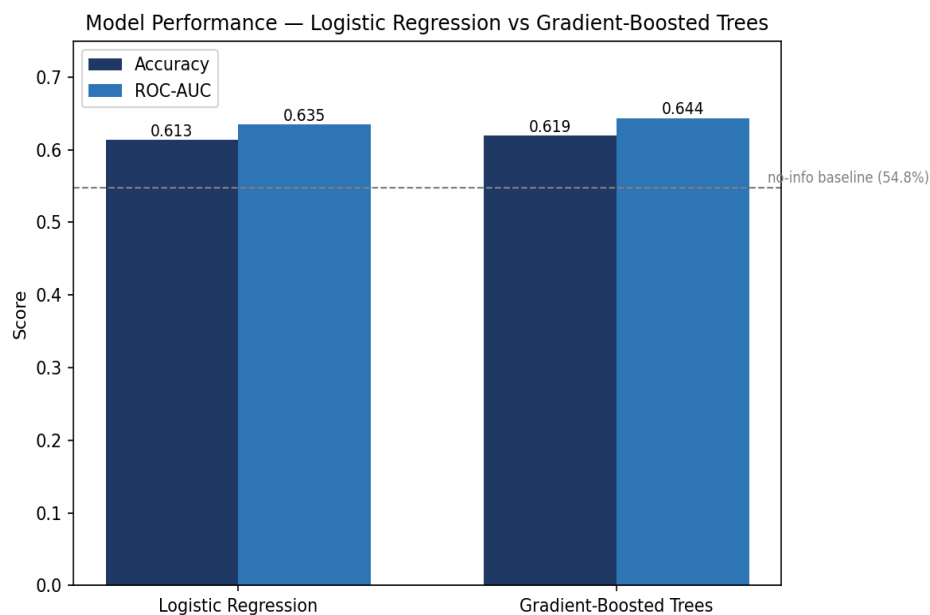
To answer this we ranked players two ways — by traditional field goal percentage, and by our shot-quality-adjusted metric — and measured how far each player moved between the two rankings.



The chart shows the players whose rankings change the most, and the pattern is clean. The biggest risers, shown in green, are perimeter shooters: Pablo Prigioni climbs about 195 spots, with Mo Williams, Jose Calderon, and Channing Frye close behind. These are players whose raw field goal percentage looks unremarkable but who consistently make difficult, contested perimeter shots the model did not expect to go in. The biggest fallers, shown in red, are interior players: Omer Asik drops about 233 spots, with Andre Drummond and others nearby. Their raw field goal percentages are high because they take easy shots near the basket, but once the model accounts for how makeable those shots were, the edge disappears. The fact that the two ends of the chart sort so cleanly by player type is the strongest evidence that the shot-quality-adjusted ranking tells a much different story than traditional efficiency metrics do.

Model performance

Underlying all of the player and team results is the shot-success model itself, so it is worth showing that the model works.



We trained two models on the same data: a logistic regression baseline and a gradient-boosted tree model. Both clear the no-information baseline of 54.8 percent — the accuracy you would get by always guessing the more common outcome — which confirms that shot context carries good predictive signal. The gradient-boosted model is the stronger of the two on both measures, reaching about 61.9 percent accuracy and a ROC-AUC of about 0.644, compared to 61.3 percent and 0.635 for the logistic baseline. The improvement is modest but consistent, which is why the boosted model is the one we use to generate the expected-make probabilities behind every other chart in this section.

Ethical Recommendations

Our project builds a model that estimates the probability any given shot goes in, then uses that model to measure which players and teams make more shots than expected. The model itself is a measurement tool, and the measurement is sound for what it does. The ethical questions don't come from the model being wrong — they come from what happens when a single number like this gets used to judge people. Shot-quality analysis feeds real decisions in the NBA: who gets signed, who gets traded, how players are paid, and who gets cut. Those are decisions about people's careers and livelihoods, and the responsible use of our work depends on understanding exactly what our metric does and does not capture.

A shot-quality metric measures one thing

The most important thing to understand about our metric is that it only measures shot-making. It asks a narrow, well-defined question — given the context of a shot, did the player make it more often than expected? — and it answers that question well. But basketball value is far wider than shot-making. A center's worth often lives in rim protection, rebounding, and setting screens. A point guard's worth lives substantially in passing, ball-handling, and running an offense. None of that appears anywhere in a shot-quality model, because the model only ever looks at shots that were taken and whether they went in. A player could be one of the most valuable in the league and still rank unremarkably on shot-quality-adjusted scoring, simply because the thing that makes them valuable is not the thing our metric measures.

This is a well-documented danger whenever a single quantitative measure is used to evaluate people. Muller (2018) describes the “tyranny of metrics” as the tendency to replace judgment built from experience with a standardized number, and then to attach real rewards and penalties to that number. The risk is not that the number is meaningless — it is that the number is treated as the whole picture when it only ever captured a slice. Thomas and Uminsky (2020) make the same argument specifically for systems built on machine learning: overemphasizing any single metric leads to a myopic and distorted view, and the appropriate response is to use a slate of metrics together rather than optimizing or evaluating on one in isolation. Our shot-quality metric is exactly the kind of single, well-optimized number their work warns against leaning on too heavily.

What our results actually show

Our analysis makes this concrete. When we rank players by shot-quality-adjusted performance and compare it to traditional field goal percentage, some players move dramatically. Low-usage perimeter players who consistently make difficult, contested shots climb the rankings, while interior players who take and make easy shots near the basket fall. It would be a mistake to read this as the metric being unfair to any group of players. The metric is doing exactly what it was designed to do: it rewards making shots that were hard to make, and it gives little credit for making shots that were easy to begin with. A rim-running center who converts high-probability layups at a solid rate ranks low on shot-quality-adjusted scoring not because the metric is broken, but because finishing easy shots is precisely the thing the metric is built to be unimpressed by.

The ethical point is not that this ranking is wrong — it is that this ranking is narrow. The same center who ranks low on our metric may be enormously valuable for reasons our model never

sees. Treating the shot-quality ranking as a verdict on that player's overall worth would badly misrepresent him, and in a context where these numbers influence contracts and roster spots, that misrepresentation could cost a player real money and opportunities.

Data limitations that affect how the results should be used

Two limitations of our data shape how far the conclusions should be pushed. First, the granular shot-tracking data that makes shot quality measurable — defender distance, shot clock, touch time, and dribbles for each individual shot — is only publicly available for the 2014-15 season. Later seasons restrict that information to aggregated summaries, so our shot-quality model is necessarily built on a single season. Any statement our analysis makes about a specific player is a statement about that player in 2014-15, not a timeless judgment, and it should always be framed that way. Second, the data identifies real, named players and makes public judgments about their shooting. These are public figures and the data is public, which is standard practice in basketball analytics, but it still means our outputs describe real people and should be presented with appropriate care.

Both sides of using the model

None of this means the model shouldn't be used. Used well, shot-quality analysis surfaces genuinely undervalued contributors — the role players who quietly make hard shots that traditional box scores overlook — and it gives teams a fairer way to recognize that kind of contribution. Choosing to ignore shot quality entirely carries its own cost, because teams then keep over-rewarding high-volume scorers who pad their numbers with easy shots and keep underpaying the players who do difficult things well. The ethical line is not whether to use the metric but how. Used as one input among many, it adds real insight. Used as a standalone verdict on a player's worth, it misleads.

Recommendations

Based on this, we recommend the following for anyone using a shot-quality model like ours to evaluate players:

First, shot-quality-adjusted scoring should always be used alongside other measures, never on its own. As Thomas and Uminsky (2020) argue, the right way to use any single metric is as one piece of a broader slate that captures the dimensions a single number cannot. For a basketball player that slate has to include defense, rebounding, passing, and the judgment of people who actually watch the games.

Second, results should always be interpreted in the context of a player's role. A low shot-quality-adjusted score means something completely different for a rim-running center than for a high-usage perimeter creator, and any responsible use of the metric has to account for what a player is actually being asked to do on the floor.

Third, anyone using these numbers in evaluation or negotiation should be transparent with players and their representatives about how they are being measured. The people who build and hold these models have an information advantage over the players whose careers the models affect, and closing that gap is part of using the tool fairly.

Finally, anyone presenting results from a model like this should be honest about its limits: that it is built on a single season of data, that it captures only shot-making and nothing else a player contributes, and that it describes a moment in time rather than a permanent truth. Presented honestly and used as one input among many, shot-quality analysis is a useful addition to how players are understood. Presented as the whole story, it would distort the very thing it is trying to measure.

Challenges

We ran into challenges in three broad areas over the course of the project: Setting up and running the data pipeline, working with the data at scale, and building the model around the limits of what the data could tell us. None of them stopped the project, but each one shaped how we worked — and are worth describing — along with how we handled it, and how we would avoid it next time.

Environment and tooling

Several of the early obstacles had nothing to do with basketball and everything to do with getting four people's machines to run the same code. The first was a Windows file-path limitation: The repository originally lived in a long directory path, and Windows' 260-character path limit caused failures when writing parquet files and installing dependencies. We solved it by moving the repository to a short path (C:\nba), which let the pipeline run correctly. The lesson for next time is simply to start the project in a short path on Windows rather than discovering the limit partway through.

A second tooling problem was conflicts between Python environments. The terminal would report that a package like 'pandas' was already installed, but importing it would still raise a `ModuleNotFoundError`; the result of a clash between a system-wide Anaconda installation and the project's virtual environment. We resolved it by standardizing on a dedicated `.venv` for everyone and installing dependencies through `python -m pip install -r requirements.txt`, which guarantees the right interpreter and package set. We also hit cross-platform differences: The gradient-boosted model depends on XGBoost, which would not load on a teammate's Mac because of a missing runtime library. Rather than ask everyone to fight their local setup, we saved the model's output tables as small CSV files in the repository and had the visualization code read those directly, so the charts can be regenerated on any machine without running the model at all. Standardizing the environment from day one, and decoupling the heavy modeling step from the lighter visualization step, are both things we would build in from the start next time.

Working with the data at scale

The full enriched dataset contained roughly 2.9 million shot records spanning 13 NBA seasons. Storing the data as Parquet files kept processing efficient even at that size, so raw volume was never the real problem. The harder part was maintaining consistent schemas across seasons and making sure our joins preserved the correct number of observations. The game data needed particular care, because the NBA API stores games at the team-game level — each game appears twice, once from each team's perspective — so joining on game alone would have duplicated shots. We joined on game, team, and season together to attach each shot to the correct team, and we ran row-count checks before and after each merge to confirm the enrichment did not silently duplicate or drop observations. In a future analysis we would hardcode those row-count assertions directly into the pipeline as automated checks rather than running them by hand.

One thing that went better than expected is worth noting as well: The underlying data was clean and well organized, and we did not encounter major data-quality errors during exploration. The rows missing critical shot-location fields (`SHOT_DISTANCE`, `LOC_X`, `LOC_Y`) were a tiny fraction of the dataset and were removed with minimal impact; dates were also standardized to a

datetime format for consistent handling across seasons. Because we did not have to spend the project fighting messy data, we could focus our limited time on the harder analytical questions.

The single-season tracking constraint

The most consequential challenge was not a bug, but a limit in the data itself. The contextual features that make shot quality measurable — defender distance, touch time, dribbles, and shot clock — are only publicly available in the Kaggle 2014-15 shot logs. The multi-season NBA API data provides shot location and outcome for every season, but not those per-shot tracking details. This meant we could not build a single model that was both rich in shot-difficulty features, and broad across many seasons.

We handled it by combining the two sources deliberately, rather than forcing one to do everything: The detailed 2014-15 shot logs power the shot-quality model behind RQ1 and RQ3, where measuring the difficulty of individual shots is essential. The full 13-season dataset powers RQ2, which is about how shot selection and success changed over time, and only needs location and outcome. This let the project balance depth and breadth: Detailed context where it existed, long-term trends where it mattered. The single-season limitation is the first thing we would address with more time, by pursuing multi-season tracking data so the model is not tied to one year.

Modeling around the data

A smaller but explicit modeling challenge was missing values in the shot-clock field of the Kaggle data. Dropping those rows would have risked pushing the sample below the project's 100,000-observation requirement, so we imputed the missing values instead, which kept the full sample of roughly 128,000 shots intact. A related, more persistent challenge was striking the right balance between engineered features and the clarity of the model. It would have been easy to keep adding derived features to chase a slightly better score, but every addition makes the model harder to explain, and for a question that is fundamentally about understanding player value, an interpretable model is worth more than a marginally more accurate one. We resolved it by keeping the feature set small and intuitive, and treating interpretability as a requirement rather than an afterthought.

We also investigated whether the model was systematically unfair to interior players, since big men who take easy shots near the rim tend to look worse once shot difficulty is accounted for. We tested a role-adjusted version of the metric that compares players within playing-style groups, and found that the role-level effect was small: The low rankings of those players reflected actual performance relative to their peers, rather than a bias in the metric. That investigation did not change the final results, but it gave us confidence in how to interpret them, and it's the kind of check we would expand into a full validation step in a longer project.

Recommendations and Next Steps

Our overall conclusion is that shot context is predictive of whether a shot goes in, and that modeling it gives us a meaningful way to separate shot-making skill from shot selection. The shot-quality lens surfaces good findings that traditional box-score metrics miss: it identifies undervalued perimeter shooters who consistently make difficult shots, it shows that the league's strategic transformation over the past decade was driven by shot selection rather than across-the-board improvement, and it reorders player efficiency in a way that is systematic and interpretable rather than arbitrary. For a team or analyst, the practical recommendation is to treat shot-quality-adjusted scoring as one input in player evaluation alongside defense, rebounding, and playmaking, where it adds the most value by flagging contributors that raw efficiency overlooks.

The more interesting question is what we would do differently if we had six to twelve months to work on this instead of the two months we had. The short timeline shaped almost every major decision we made, and a longer horizon would change the project in a few specific ways.

The first and most important change would be the data. The single biggest constraint we faced is that the per-shot tracking features that make shot quality measurable — defender distance, shot clock, touch time, and dribbles — are only freely available for the 2014-15 season, which forced us to build the shot-quality model on a single year. With more time, the data itself becomes the project's first phase rather than a fixed starting point. We could reconstruct multi-season tracking data from the aggregated splits the NBA does publish, pursue academic or licensed access to the full tracking feed, or build a longer-running collection pipeline against the sources that do exist. Having those features across many seasons is what unlocks everything else: a model that is not tied to a single year, the ability to test whether our findings hold over time, and the chance to answer all three research questions on the same dataset rather than splitting between a single-season model and multi-season location data.

With multi-season tracking data in hand, the second change would be to the modeling. Two months was enough to build a solid gradient-boosted model and validate it, but it was not enough to pursue the richer spatial and sequential approaches from the literature: The expected-possession-value style models that treat a possession as it unfolds rather than scoring shots in isolation. Those methods require considerably more engineering and tuning time, and with a longer runway we would build toward them, along with proper cross-season validation and checks that a two-month project simply cannot fit.

The third change would be to broaden what the metric measures. The clearest limitation of our work, which we discuss in the ethical recommendations, is that a shot-quality model only captures shot-making and says nothing about the defense, rebounding, and playmaking that make up most of a player's value. With six to twelve months we would treat that as an objective rather than a caveat: Building a more complete player value framework that combines shot quality with the other dimensions of the game, and developing the role-aware or position-adjusted version of the metric that would let us compare players fairly against others who take similar shots. We would also validate our playing-style archetypes against an independent set of role labels to confirm they reflect real basketball roles. Each of these is a natural extension of what we have already built; what we lacked was not the idea but the time.

References

Data and software

- Becker, D. (2015). *NBA shot logs* [Data set]. Kaggle.
<https://www.kaggle.com/dansbecker/nba-shot-logs>
- Patel, S. (n.d.). *nba_api: NBA Stats API Python wrapper* [Computer software]. GitHub.
https://github.com/swar/nba_api
- sumitrodatta. (2024). *NBA stats (1947–present)* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/sumitrodatta/nba-aba-baa-stats>

Literature

- Bajons, R., & Kook, L. (2025). *Rethinking player evaluation in sports: Goals above expectation and beyond* (arXiv:2509.20083) [Preprint]. arXiv. <https://arxiv.org/abs/2509.20083>
- Barocas, S., Hardt, M., & Narayanan, A. (2023). *Fairness and machine learning: Limitations and opportunities*. MIT Press. <https://fairmlbook.org>
- Cervone, D., D'Amour, A., Bornn, L., & Goldsberry, K. (2014). *A multiresolution stochastic process model for predicting basketball possession outcomes* (arXiv:1408.0777) [Preprint]. arXiv. <https://arxiv.org/abs/1408.0777>
- Cervone, D., D'Amour, A., Bornn, L., & Goldsberry, K. (2014). POINTWISE: Predicting points and valuing decisions in real time with NBA optical tracking data. *MIT Sloan Sports Analytics Conference*.
- Expected points above average: A novel NBA player metric based on Bayesian hierarchical modeling*. (2024). (arXiv:2405.10453) [Preprint]. arXiv. <https://arxiv.org/abs/2405.10453>
- Kono, R., & Fujii, K. (2024). *Mathematical models for off-ball scoring prediction in basketball* (arXiv:2406.08749) [Preprint]. arXiv. <https://arxiv.org/abs/2406.08749>
- Muller, J. Z. (2018). *The tyranny of metrics*. Princeton University Press.
- Oliver, D. (2004). *Basketball on paper: Rules and tools for performance analysis*. Brassey's.
- Poropudas, J., & Halme, T. (2023). *Dean Oliver's four factors revisited* (arXiv:2305.13032) [Preprint]. arXiv. <https://arxiv.org/abs/2305.13032>
- Skinner, B. (2011). *The problem of shot selection in basketball* (arXiv:1107.5793) [Preprint]. arXiv. <https://arxiv.org/abs/1107.5793>
- Thomas, R. L., & Uminsky, D. (2020). *Reliance on metrics is a fundamental challenge for AI* (arXiv:2002.08512) [Preprint]. arXiv. <https://arxiv.org/abs/2002.08512>

Appendix: Source Code

The source code below implements the data pipeline, feature engineering, and modeling for the project. It is included here per the assignment requirement; the interactive notebooks are not included.

src/pulls/_paths.py

```
"""Shared paths for the pull scripts.

One place to set where our raw data ends up, so the individual modules
don't each reinvent it.
"""
from __future__ import annotations
from pathlib import Path

# Repo root (this file is at src/pulls/_paths.py, so go up 3 levels)
REPO_ROOT = Path(__file__).resolve().parents[2]

DATA_ROOT = REPO_ROOT / "data"
RAW = DATA_ROOT / "raw"

SHOTS_DIR = RAW / "shots"
GAMES_DIR = RAW / "games"
PLAYERS_FILE = RAW / "players.parquet"
TEAMS_FILE = RAW / "teams.parquet"
KAGGLE_DIR = RAW / "kaggle"

def ensure_dirs() -> None:
    """Make sure all the raw data subdirectories exist."""
    for d in (SHOTS_DIR, GAMES_DIR, KAGGLE_DIR):
        d.mkdir(parents=True, exist_ok=True)

# Default season range: 2013-14 (start of the tracking-data era) through current.
# Bump DEFAULT_END_SEASON every year as new seasons wrap up.
DEFAULT_START_SEASON = 2013
DEFAULT_END_SEASON = 2025 # the 2025-26 season

def season_str(start_year: int) -> str:
    """Format a season the NBA way: 'YYYY-YY'."""

    >>> season_str(2023)
    '2023-24'
    """
    return f"{start_year}-{str(start_year + 1)[-2:]}"

def all_seasons(start: int = DEFAULT_START_SEASON, end: int = DEFAULT_END_SEASON) -> list[str]:
    """Give back all season strings from start to end, inclusive."""
    return [season_str(y) for y in range(start, end + 1)]
```

src/pulls/nba_teams.py

```
"""Pull the list of NBA teams.

Teams are static metadata that comes bundled with nba_api, so this doesn't
actually hit the API – it just reads from memory. Output: data/raw/teams.parquet
"""
from __future__ import annotations
import pandas as pd
```

```

from nba_api.stats.static import teams

from src.pulls._paths import TEAMS_FILE, ensure_dirs

def fetch_teams() -> pd.DataFrame:
    """Give back a DataFrame of every NBA team."""
    return pd.DataFrame(teams.get_teams())

def main() -> None:
    ensure_dirs()
    df = fetch_teams()
    df.to_parquet(TEAMS_FILE, index=False)
    print(f"Wrote {len(df)} teams to {TEAMS_FILE}")

if __name__ == "__main__":
    main()

```

src/pulls/nba_players.py

"""Pull every NBA player, historical + current.

Uses CommonAllPlayers with is_only_current_season=0 so we get every player who's ever played in an NBA game. Output: data/raw/players.parquet

```

from __future__ import annotations
import time
import pandas as pd
from nba_api.stats.endpoints import commonallplayers

from src.pulls._paths import PLAYERS_FILE, ensure_dirs

```

```

def fetch_players() -> pd.DataFrame:
    """Give back a DataFrame of every NBA player, past and present."""
    endpoint = commonallplayers.CommonAllPlayers(
        is_only_current_season=0,
        league_id="00", # NBA
    )
    return endpoint.get_data_frames()[0]

```

```

def main() -> None:
    ensure_dirs()
    print("Fetching all NBA players (historical + current)...")
    df = fetch_players()
    df.to_parquet(PLAYERS_FILE, index=False)
    print(f"Wrote {len(df)} players to {PLAYERS_FILE}")

```

```

if __name__ == "__main__":
    main()

```

src/pulls/nba_games.py

"""Pull the league-wide game logs, season by season.

Uses LeagueGameLog, which gives us one row per (team, game). We grab both Regular Season and Playoffs for each season. Idempotent – already-pulled seasons get skipped.

Output: data/raw/games/games_{SEASON}.parquet (e.g. games_2023-24.parquet)

```

"""
from __future__ import annotations
import time
import pandas as pd
from nba_api.stats.endpoints import leaguegameblog

from src.pulls._paths import GAMES_DIR, all_seasons, ensure_dirs

# Sleep between API calls so we don't get rate-limited – NBA.com is strict
SLEEP_SECONDS = 0.6

def fetch_season_games(season: str) -> pd.DataFrame:
    """Pull all the games (regular + playoffs) for one season."""
    frames = []
    for season_type in ("Regular Season", "Playoffs"):
        try:
            endpoint = leaguegameblog.LeagueGameLog(
                season=season,
                season_type_all_star=season_type,
                league_id="00",
            )
            df = endpoint.get_data_frames()[0]
            df["SEASON_TYPE"] = season_type
            frames.append(df)
        except Exception as exc: # broad catch – the API can fail in weird ways
            print(f" Warning: {season} {season_type} failed: {exc}")
            time.sleep(SLEEP_SECONDS)
    if not frames:
        return pd.DataFrame()
    return pd.concat(frames, ignore_index=True)

def main(seasons: list[str] | None = None) -> None:
    ensure_dirs()
    seasons = seasons or all_seasons()
    for season in seasons:
        out = GAMES_DIR / f"games_{season}.parquet"
        if out.exists():
            print(f"Skip {season} (already cached at {out.name})")
            continue
        print(f"Fetching games for {season}...")
        df = fetch_season_games(season)
        if df.empty:
            print(f" No data returned for {season}; skipping write")
            continue
        df.to_parquet(out, index=False)
        print(f" Wrote {len(df)} rows to {out.name}")

if __name__ == "__main__":
    main()

```

src/pulls/nba_shots.py

"""Pull the shot-detail data, season by season (this is our main dataset).

How it works: we walk every team within every season and grab that team's shots for both Regular Season and Playoffs. ShotChartDetail gives us shot-level rows with court coordinates, shot zone, distance, and outcome.

This is the slowest pull we have – about 30 teams x 2 season types per season, with a polite sleep between each call. Figure ~30-40 seconds per season, and the full 2013-present pull takes around 8-10 minutes.

Idempotent – already-pulled seasons get skipped.


```

Output: data/raw/shots/shots_{SEASON}.parquet (e.g. shots_2023-24.parquet)
"""
from __future__ import annotations
import time
import pandas as pd
from nba_api.stats.static import teams as static_teams
from nba_api.stats.endpoints import shotchartdetail

from src.pulls._paths import SHOTS_DIR, all_seasons, ensure_dirs

SLEEP_SECONDS = 0.6

def fetch_team_season_shots(team_id: int, season: str, season_type: str) -> pd.DataFrame:
    """Grab one team's shots for one season + season type."""
    endpoint = shotchartdetail.ShotChartDetail(
        team_id=team_id,
        player_id=0, # 0 means all players on the team
        season_nullable=season,
        season_type_all_star=season_type,
        context_measure_simple="FGA",
    )
    return endpoint.get_data_frames()[0]

def fetch_season_shots(season: str) -> pd.DataFrame:
    """Pull every shot for one season by walking through the teams."""
    team_ids = [t["id"] for t in static_teams.get_teams()]
    frames = []
    for i, team_id in enumerate(team_ids, 1):
        for season_type in ("Regular Season", "Playoffs"):
            try:
                df = fetch_team_season_shots(team_id, season, season_type)
                if not df.empty:
                    df["SEASON_TYPE"] = season_type
                    frames.append(df)
            except Exception as exc:
                print(f" Warning: team {team_id} {season} {season_type} failed: {exc}")
                time.sleep(SLEEP_SECONDS)
        print(f" [{i}/{len(team_ids)}] team {team_id} done")
    if not frames:
        return pd.DataFrame()
    return pd.concat(frames, ignore_index=True)

def main(seasons: list[str] | None = None) -> None:
    ensure_dirs()
    seasons = seasons or all_seasons()
    for season in seasons:
        out = SHOTS_DIR / f"shots_{season}.parquet"
        if out.exists():
            print(f"Skip {season} (already cached at {out.name})")
            continue
        print(f"Fetching shots for {season}...")
        df = fetch_season_shots(season)
        if df.empty:
            print(f" No data returned for {season}; skipping write")
            continue
        df.to_parquet(out, index=False)
        print(f" Wrote {len(df)} rows to {out.name}")

if __name__ == "__main__":
    main()

```

src/pulls/kaggle_datasets.py

```

"""Download our two supporting Kaggle datasets.

```

Before this works:

1. `pip install kaggle` (already in requirements.txt)
2. A Kaggle API token at ~/.kaggle/kaggle.json (Windows: %USERPROFILE%\kaggle\kaggle.json)
Grab yours from kaggle.com -> account settings -> "Create New API Token"

Each dataset lands in its own subfolder under data/raw/kaggle/.
Idempotent – already-downloaded datasets get skipped.

If one of the slugs below stops working (Kaggle URLs can change),
look up the current one at kaggle.com and update the constant.

```
"""  
from __future__ import annotations  
from pathlib import Path
```

```
from src.pulls._paths import KAGGLE_DIR, ensure_dirs
```

```
# Dataset slugs (the "owner/dataset" part of the Kaggle URL).  
# Both are current as of project setup. If one 404s, look up the  
# current slug at kaggle.com and swap it in.  
SHOT_LOGS_SLUG = "dansbecker/nba-shot-logs" # 128k shots, 2014-15 only (for prototyping)  
HISTORICAL_STATS_SLUG = "sumitrodatta/nba-aba-baa-stats" # 1947-present, actively maintained
```

```
DATASETS = {  
    "shot_logs_2014_15": SHOT_LOGS_SLUG,  
    "historical_stats": HISTORICAL_STATS_SLUG,  
}
```

```
def _has_extracted_files(dest: Path) -> bool:  
    """True if the folder exists and has at least one non-hidden file in it."""  
    if not dest.exists():  
        return False  
    return any(p.is_file() and not p.name.startswith(".") for p in dest.iterdir())
```

```
def download_dataset(slug: str, dest: Path) -> None:  
    """Download and unzip one Kaggle dataset into `dest`."""  
    # Import here so the module can be loaded without Kaggle creds being set up  
    # (handy for inspection); creds only matter at actual download time.  
    from kaggle import api as kaggle_api # type: ignore  
  
    dest.mkdir(parents=True, exist_ok=True)  
    print(f" Downloading {slug} -> {dest}")  
    kaggle_api.dataset_download_files(slug, path=str(dest), unzip=True, quiet=False)
```

```
def main() -> None:  
    ensure_dirs()  
    for name, slug in DATASETS.items():  
        dest = KAGGLE_DIR / name  
        if _has_extracted_files(dest):  
            print(f"Skip {name} (already extracted at {dest})")  
            continue  
        try:  
            download_dataset(slug, dest)  
            print(f" Done: {name}")  
        except Exception as exc:  
            print(f" FAILED: {name} ({slug}) -> {exc}")  
            print(f" If that's a 404, check the slug at kaggle.com and update kaggle_datasets.py")
```

```
if __name__ == "__main__":  
    main()
```

src/pulls/setup_kaggle.py

"""One-shot helper to set up Kaggle credentials.

Run this once per machine. It'll prompt you for your Kaggle username and API key, write them to the right place, set the right permissions, and test that the kaggle CLI can authenticate.

Usage:

```
python -m src.pulls.setup_kaggle
```

Where to get your username + key:

1. Go to <https://www.kaggle.com/settings>
2. Scroll to the "API" section
3. Click "Create New API Token"
4. A box pops up showing your username and key (or downloads a kaggle.json file with both values inside).

If Kaggle downloaded a kaggle.json file and you don't want to retype the values, you can just open that file in Notepad to see them, then paste them here when prompted.

"""

```
from __future__ import annotations
import json
import os
import stat
from pathlib import Path
```

```
def main() -> None:
    print("=" * 60)
    print("Kaggle credentials setup")
    print("=" * 60)
    print()
    print("This will create ~/.kaggle/kaggle.json so the project's")
    print("Kaggle pulls can authenticate as you.")
    print()
    print("If you've never created an API token before, go to:")
    print("    https://www.kaggle.com/settings")
    print("Scroll to the 'API' section -> click 'Create New API Token'.")
    print()

    username = input("Kaggle username: ").strip()
    if not username:
        print("ERROR: username cannot be empty. Aborting.")
        return

    key = input("Kaggle API key: ").strip()
    if not key:
        print("ERROR: key cannot be empty. Aborting.")
        return

    # Find the right folder. ~/.kaggle/ on every OS.
    kaggle_dir = Path.home() / ".kaggle"
    kaggle_dir.mkdir(exist_ok=True)
    creds_path = kaggle_dir / "kaggle.json"

    if creds_path.exists():
        ans = input(f"\n{creds_path} already exists. Overwrite? [y/N]: ").strip().lower()
        if ans != "y":
            print("Aborted. Existing file untouched.")
            return

    # Write the credentials. Kaggle expects a single-line JSON object.
    with creds_path.open("w", encoding="utf-8") as f:
        json.dump({"username": username, "key": key}, f)
```

```

# On Linux/Mac, the kaggle CLI complains if the file is world-readable.
# On Windows this is a no-op but doesn't hurt.
try:
    creds_path.chmod(stat.S_IRUSR | stat.S_IWUSR)
except Exception:
    pass

print(f"\nWrote credentials to: {creds_path}")
print()

# Verify by asking the kaggle library to authenticate.
print("Testing authentication...")
try:
    from kaggle import api as kaggle_api # type: ignore
    kaggle_api.authenticate()
    print(" SUCCESS - kaggle CLI can authenticate as you.")
    print()
    print("You're set. To run the Kaggle dataset downloads:")
    print("    python -m src.pulls.kaggle_datasets")
    print()
    print("Or to run every data pull in order:")
    print("    python -m src.pulls.fetch_all")
except Exception as exc:
    print(f" FAILED: {exc}")
    print()
    print("Double-check that the username and key you entered match what")
    print("Kaggle shows under https://www.kaggle.com/settings -> API.")
    print(f"If you need to retry, delete {creds_path} and re-run this script.")

if __name__ == "__main__":
    main()

```

src/pulls/fetch_all.py

"""Run every pull in the right order.

Usage:

```
python -m src.pulls.fetch_all
```

Each pull is idempotent, so re-running is safe. If you want to force a re-pull for a specific season, delete that season's parquet file first.

Total runtime on a fresh machine: ~10-15 minutes (most of it is the shot pull).

```

"""
from __future__ import annotations
import time

from src.pulls import nba_teams, nba_players, nba_games, nba_shots, kaggle_datasets

def main() -> None:
    start = time.time()
    print("=" * 60)
    print("Step 1/5: NBA teams (static metadata, instant)")
    print("=" * 60)
    nba_teams.main()

    print("\n" + "=" * 60)
    print("Step 2/5: NBA players (one API call)")
    print("=" * 60)
    nba_players.main()

    print("\n" + "=" * 60)
    print("Step 3/5: Game logs (~30 seconds per season)")
    print("=" * 60)
    nba_games.main()

```

```

print("\n" + "=" * 60)
print("Step 4/5: Shot detail (~30-40 seconds per season – slowest step)")
print("=" * 60)
nba_shots.main()

print("\n" + "=" * 60)
print("Step 5/5: Kaggle datasets (needs ~/.kaggle/kaggle.json)")
print("=" * 60)
kaggle_datasets.main()

elapsed = time.time() - start
print(f"\nAll pulls done in {elapsed/60:.1f} minutes.")

if __name__ == "__main__":
    main()

```

src/clean/shots.py

```

"""
Loads raw shot data, performs cleaning and standardization,
and writes a cleaned shot table to data/interim/shots_clean.parquet.
"""

from pathlib import Path
import pandas as pd

RAW_DIR = Path("data/raw/shots")
OUTPUT_FILE = Path("data/interim/shots_clean.parquet")

def load_shots():
    frames = []

    for file in sorted(RAW_DIR.glob("shots_*.parquet")):
        print(f>Loading {file.name}")

        df = pd.read_parquet(file)

        df["SEASON"] = file.stem.replace("shots_", "")

        frames.append(df)

    return pd.concat(frames, ignore_index=True)

def clean_shots(df):
    df = df.drop_duplicates()

    df = df.dropna(
        subset=[
            "SHOT_DISTANCE",
            "LOC_X",
            "LOC_Y"
        ]
    )

    df["GAME_DATE"] = pd.to_datetime(
        df["GAME_DATE"],
        errors="coerce"
    )

    return df

```

```

def main():
    shots = load_shots()

    print("Raw shape:", shots.shape)

    shots = clean_shots(shots)

    print("Clean shape:", shots.shape)

    OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)

    shots.to_parquet(OUTPUT_FILE, index=False)

    print(f"Saved to {OUTPUT_FILE}")

if __name__ == "__main__":
    main()

```

src/clean/build_enriched.py

```

"""
Builds an enriched shot-level dataset by joining shots,
players, teams, and games data and writes the result
to data/processed/shots_enriched.parquet.
"""

from pathlib import Path
import pandas as pd

SHOTS_FILE = Path("data/interim/shots_clean.parquet")
PLAYERS_FILE = Path("data/raw/players.parquet")
TEAMS_FILE = Path("data/raw/teams.parquet")
GAMES_DIR = Path("data/raw/games")
OUTPUT_FILE = Path("data/processed/shots_enriched.parquet")

def load_games():
    frames = []

    for file in sorted(GAMES_DIR.glob("games_*.parquet")):
        print(f>Loading {file.name}")
        df = pd.read_parquet(file)
        df["SEASON"] = file.stem.replace("games_", "")
        frames.append(df)

    return pd.concat(frames, ignore_index=True)

def main():
    shots = pd.read_parquet(SHOTS_FILE)
    players = pd.read_parquet(PLAYERS_FILE)
    teams = pd.read_parquet(TEAMS_FILE)
    games = load_games()

    players_small = players[
        ["PERSON_ID", "DISPLAY_FIRST_LAST", "FROM_YEAR", "TO_YEAR"]
    ].drop_duplicates()

    teams_small = teams[
        ["id", "full_name", "abbreviation", "city", "state", "year_founded"]
    ].drop_duplicates()

```

```

games_small = games[
    ["GAME_ID", "TEAM_ID", "SEASON", "MATCHUP", "WL", "PTS", "PLUS_MINUS"]
].drop_duplicates()

enriched = shots.merge(
    players_small,
    left_on="PLAYER_ID",
    right_on="PERSON_ID",
    how="left"
).drop(columns=["PERSON_ID"])

enriched = enriched.merge(
    teams_small,
    left_on="TEAM_ID",
    right_on="id",
    how="left"
).drop(columns=["id"])

enriched = enriched.merge(
    games_small,
    on=["GAME_ID", "TEAM_ID", "SEASON"],
    how="left"
)

OUTPUT_FILE.parent.mkdir(parents=True, exist_ok=True)

enriched.to_parquet(OUTPUT_FILE, index=False)

print("Shots shape:", shots.shape)
print("Enriched shape:", enriched.shape)
print(f"Saved to {OUTPUT_FILE}")

if __name__ == "__main__":
    main()

```

src/features/kaggle_shot_quality.py

"""Reusable helpers for the Kaggle 2014-15 shot-quality models.

The point of this module is to keep the modeling notebook focused on analysis instead of repeating the same cleanup and feature logic.

```

from __future__ import annotations

import re
from pathlib import Path

import numpy as np
import pandas as pd

from src.pulls._paths import REPO_ROOT

KAGGLE_SHOT_LOG = REPO_ROOT / "data" / "raw" / "kaggle" / "shot_logs_2014_15" / "shot_logs.csv"

TARGET_COLUMN = "SHOT_MADE"
PLAYER_NAME_COLUMN = "player_name"
TEAM_COLUMN = "SHOOTING_TEAM"

NUMERIC_FEATURES = [
    "FINAL_MARGIN",
    "SHOT_NUMBER",
    "PERIOD",

```

```

"SHOT_CLOCK",
"DRIBBLES",
"TOUCH_TIME",
"SHOT_DIST",
"CLOSE_DEF_DIST",
"GAME_CLOCK_SECONDS",
]

CATEGORICAL_FEATURES = [
    "LOCATION",
    "W",
    "PTS_TYPE",
    "SHOT_DIST_ZONE",
    "TOUCH_TIME_BUCKET",
    "LATE_CLOCK",
]

MODEL_COLUMNS = NUMERIC_FEATURES + CATEGORICAL_FEATURES

MATCHUP_PATTERN = re.compile(r"\w{3} \d{2}, \d{4} - ([A-Z]{2,3}) (?:@|vs\.)([A-Z]{2,3})")

def load_kaggle_shot_logs(csv_path: Path | None = None) -> pd.DataFrame:
    """Load the 2014-15 Kaggle shot log dataset."""
    path = csv_path or KAGGLE_SHOT_LOG
    return pd.read_csv(path)

def game_clock_to_seconds(clock_value: str) -> float:
    """Convert MM:SS game clock strings into seconds remaining in the period."""
    if pd.isna(clock_value):
        return np.nan

    minutes, seconds = str(clock_value).split(":")
    return int(minutes) * 60 + int(seconds)

def extract_shooting_team(matchup: str, location: str) -> str | None:
    """Recover the shooting team abbreviation from the matchup string.

    Kaggle stores the matchup as text like ``CHA @ BKN`` or ``CHA vs. LAL``.
    We use the home/away flag to back out which team took the shot.
    """
    if pd.isna(matchup) or pd.isna(location):
        return None

    match = MATCHUP_PATTERN.search(str(matchup))
    if not match:
        return None

    away_team, home_team = match.groups()
    return home_team if location == "H" else away_team

def add_derived_features(shots: pd.DataFrame) -> pd.DataFrame:
    """Add the reusable shot-quality features used across the models."""
    enriched = shots.copy()

    enriched["GAME_CLOCK_SECONDS"] = enriched["GAME_CLOCK"].map(game_clock_to_seconds)
    enriched[TARGET_COLUMN] = (enriched["SHOT_RESULT"] == "made").astype(int)
    enriched[TEAM_COLUMN] = [
        extract_shooting_team(matchup, location)
        for matchup, location in zip(enriched["MATCHUP"], enriched["LOCATION"], strict=False)
    ]

```



```

# A few human-readable buckets make the residual analysis easier to explain.
enriched["SHOT_DIST_ZONE"] = pd.cut(
    enriched["SHOT_DIST"],
    bins=[-np.inf, 4, 14, 23.75, np.inf],
    labels=["at_rim", "short_midrange", "long_midrange", "three_plus"],
).astype("object")

enriched["TOUCH_TIME_BUCKET"] = pd.cut(
    enriched["TOUCH_TIME"],
    bins=[-np.inf, 2, 6, np.inf],
    labels=["quick", "balanced", "hold"],
).astype("object")

enriched["LATE_CLOCK"] = np.where(enriched["SHOT_CLOCK"].fillna(24) <= 4, "late_clock", "normal_clock")

return enriched

def build_modeling_frame(shots: pd.DataFrame) -> pd.DataFrame:
    """Return the feature-ready table used by the notebook models.

    We keep the row filter intentionally light so the sample stays above
    the course's 100K observation floor.
    """
    modeling_frame = add_derived_features(shots)
    modeling_frame = modeling_frame.dropna(subset=["SHOT_RESULT", PLAYER_NAME_COLUMN])
    return modeling_frame

def build_residual_table(
    scored_shots: pd.DataFrame,
    group_column: str,
    min_shots: int,
) -> pd.DataFrame:
    """Aggregate actual vs expected shooting results for players or teams."""
    summary = (
        scored_shots.groupby(group_column, dropna=False)
        .agg(
            shots=(TARGET_COLUMN, "size"),
            actual_makes=(TARGET_COLUMN, "sum"),
            expected_makes=("EXPECTED_MAKE_PROB", "sum"),
            actual_fg_pct=(TARGET_COLUMN, "mean"),
            expected_fg_pct=("EXPECTED_MAKE_PROB", "mean"),
        )
        .reset_index()
    )

    # Small samples can jump around a lot, so trim them before ranking.
    summary = summary[summary["shots"] >= min_shots].copy()
    summary["makes_above_expected"] = summary["actual_makes"] - summary["expected_makes"]
    summary["fg_pct_above_expected"] = summary["actual_fg_pct"] - summary["expected_fg_pct"]
    summary = summary.sort_values("makes_above_expected", ascending=False).reset_index(drop=True)

    return summary

```

src/models/build_model_outputs.py

```

"""Fit the shot-quality models and write their output tables to disk.

```

This script runs the logistic and gradient-boosted models on the Kaggle 2014-15 shot logs and saves the player residuals, team residuals, rank table, and model metrics as CSV files under ``data/model\_outputs/``.

Running the models needs XGBoost, so this step is run once on a machine where XGBoost is available. The visualization notebook then reads the

saved CSVs directly, which means the charts can be regenerated on any machine without having to fit the models again.

Usage:

```
python -m src.models.build_model_outputs
```

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
import pandas as pd
from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from xgboost import XGBClassifier
```

```
from src.features.kaggle_shot_quality import (
    CATEGORICAL_FEATURES,
    MODEL_COLUMNS,
    NUMERIC_FEATURES,
    PLAYER_NAME_COLUMN,
    TARGET_COLUMN,
    TEAM_COLUMN,
    build_modeling_frame,
    build_residual_table,
    load_kaggle_shot_logs,
)
from src.pulls._paths import REPO_ROOT
```

```
OUTPUT_DIR = REPO_ROOT / "data" / "model_outputs"
```

```
PLAYER_MIN_SHOTS = 150
TEAM_MIN_SHOTS = 2000
```

```
def build_logistic_model() -> Pipeline:
    """Logistic regression with scaled numeric and one-hot categorical inputs."""
    preprocessor = ColumnTransformer([
        (
            "num",
            Pipeline([
                ("imputer", SimpleImputer(strategy="median")),
                ("scaler", StandardScaler()),
            ]),
            NUMERIC_FEATURES,
        ),
        (
            "cat",
            Pipeline([
                ("imputer", SimpleImputer(strategy="most_frequent")),
                ("onehot", OneHotEncoder(handle_unknown="ignore")),
            ]),
            CATEGORICAL_FEATURES,
        ),
    ])
    return Pipeline([
        ("preprocessor", preprocessor),
        ("classifier", LogisticRegression(max_iter=1000, solver="lbfgs")),
    ])
```

```

def build_boosted_model() -> Pipeline:
    """Gradient-boosted trees with imputed numeric and one-hot categorical inputs."""
    preprocessor = ColumnTransformer([
        (
            "num",
            Pipeline([("imputer", SimpleImputer(strategy="median"))]),
            NUMERIC_FEATURES,
        ),
        (
            "cat",
            Pipeline([
                ("imputer", SimpleImputer(strategy="most_frequent")),
                ("onehot", OneHotEncoder(handle_unknown="ignore")),
            ]),
            CATEGORICAL_FEATURES,
        ),
    ])
    return Pipeline([
        ("preprocessor", preprocessor),
        ("classifier", XGBClassifier(
            objective="binary:logistic",
            eval_metric="logloss",
            n_estimators=300,
            max_depth=5,
            learning_rate=0.05,
            subsample=0.8,
            colsample_bytree=0.8,
            min_child_weight=5,
            reg_lambda=1.0,
            random_state=42,
            tree_method="hist",
            n_jobs=4,
        )),
    ])

def score_models(modeling_shots: pd.DataFrame) -> pd.DataFrame:
    """Train both models on a hold-out split and return their test metrics."""
    features = modeling_shots[MODEL_COLUMNS].copy()
    outcomes = modeling_shots[TARGET_COLUMN].copy()

    X_train, X_test, y_train, y_test = train_test_split(
        features,
        outcomes,
        test_size=0.2,
        random_state=42,
        stratify=outcomes,
    )

    rows = []
    for name, model in [("Logistic Regression", build_logistic_model()),
                        ("Gradient-Boosted Trees", build_boosted_model())]:
        trained = clone(model)
        trained.fit(X_train, y_train)
        predicted_labels = trained.predict(X_test)
        predicted_probs = trained.predict_proba(X_test)[: , 1]
        rows.append({
            "model": name,
            "accuracy": accuracy_score(y_test, predicted_labels),
            "roc_auc": roc_auc_score(y_test, predicted_probs),
        })

    return pd.DataFrame(rows)

def build_rank_table(scored_shots: pd.DataFrame) -> pd.DataFrame:
    """Compare each player's traditional efficiency to a shot-quality-adjusted view."""
    rank_table = (
        scored_shots.groupby(PPLAYER_NAME_COLUMN)

```

```

        .agg(
            shots=(TARGET_COLUMN, "size"),
            fgm=("FGM", "sum"),
            points=("PTS", "sum"),
            expected_makes=("EXPECTED_MAKE_PROB", "sum"),
        )
        .reset_index()

    )

    rank_table = rank_table[rank_table["shots"] >= PLAYER_MIN_SHOTS].copy()
    rank_table["fg_pct"] = rank_table["fgm"] / rank_table["shots"]
    rank_table["efg_pct"] = (rank_table["points"] / 2) / rank_table["shots"]
    rank_table["expected_fg_pct"] = rank_table["expected_makes"] / rank_table["shots"]
    rank_table["shot_quality_adjusted"] = rank_table["fg_pct"] - rank_table["expected_fg_pct"]

    rank_table["fg_rank"] = rank_table["fg_pct"].rank(ascending=False, method="min")
    rank_table["efg_rank"] = rank_table["efg_pct"].rank(ascending=False, method="min")
    rank_table["adjusted_rank"] = rank_table["shot_quality_adjusted"].rank(ascending=False, method="min")
    rank_table["rank_shift_vs_fg"] = rank_table["fg_rank"] - rank_table["adjusted_rank"]
    rank_table["rank_shift_vs_efg"] = rank_table["efg_rank"] - rank_table["adjusted_rank"]

    return rank_table.sort_values("rank_shift_vs_fg", ascending=False).reset_index(drop=True)

def main() -> None:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    modeling_shots = build_modeling_frame(load_kaggle_shot_logs())
    print(f"Modeling rows: {len(modeling_shots):,}")

    metrics = score_models(modeling_shots)
    print("\nModel metrics:")
    print(metrics.to_string(index=False))

    # Refit the boosted model on every shot so each one gets an expected make
    # probability we can compare against what actually happened.
    full_model = build_boosted_model()
    full_model.fit(modeling_shots[MODEL_COLUMNS], modeling_shots[TARGET_COLUMN])

    scored_shots = modeling_shots[
        [PLAYER_NAME_COLUMN, TEAM_COLUMN, TARGET_COLUMN, "FGM", "PTS"]
    ].copy()
    scored_shots["EXPECTED_MAKE_PROB"] = full_model.predict_proba(modeling_shots[MODEL_COLUMNS])[:, 1]

    player_residuals = build_residual_table(scored_shots, PLAYER_NAME_COLUMN, min_shots=PLAYER_MIN_SHOTS)
    team_residuals = build_residual_table(scored_shots, TEAM_COLUMN, min_shots=TEAM_MIN_SHOTS)
    rank_table = build_rank_table(scored_shots)

    outputs = {
        "player_residuals.csv": player_residuals,
        "team_residuals.csv": team_residuals,
        "rank_table.csv": rank_table,
        "model_metrics.csv": metrics,
    }
    for filename, frame in outputs.items():
        path = OUTPUT_DIR / filename
        frame.to_csv(path, index=False)
        print(f"Wrote {len(frame):>4} rows to {path.relative_to(REPO_ROOT)}")

if __name__ == "__main__":
    main()

```

src/models/build_analysis_outputs.py

"""Build the remaining RQ1 and RQ3 analysis tables and save them to disk.

This script covers the two analysis pieces that sit on top of the shot-quality model:

- * RQ1 - group the overperforming players into playing-style archetypes using K-means clustering, so we can describe *what distinguishes* the players who beat their expected shot value.
- * RQ3 - measure how much the shot-quality-adjusted ranking differs from the traditional efficiency metrics using Spearman rank correlation.

It reads the player residuals saved by ``build\_model\_outputs`` and the Kaggle shot logs, so it does not need to refit any model and runs without XGBoost.

Usage:

```
python -m src.models.build_analysis_outputs

from __future__ import annotations

import pandas as pd
from scipy.stats import spearmanr
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

from src.features.kaggle_shot_quality import (
    PLAYER_NAME_COLUMN,
    TARGET_COLUMN,
    build_modeling_frame,
    load_kaggle_shot_logs,
)
from src.pulls._paths import REPO_ROOT

OUTPUT_DIR = REPO_ROOT / "data" / "model_outputs"

STYLE_FEATURES = [
    "avg_shot_dist",
    "avg_def_dist",
    "avg_touch_time",
    "avg_dribbles",
    "three_rate",
]
MIN_SHOTS = 150
N_ARCHETYPES = 4

def build_player_style(shots: pd.DataFrame) -> pd.DataFrame:
    """Summarize each player's shot diet into playing-style features."""
    style = (
        shots.groupby(PLAYER_NAME_COLUMN)
        .agg(
            shots=(TARGET_COLUMN, "size"),
            avg_shot_dist=("SHOT_DIST", "mean"),
            avg_def_dist=("CLOSE_DEF_DIST", "mean"),
            avg_touch_time=("TOUCH_TIME", "mean"),
            avg_dribbles=("DRIBBLES", "mean"),
            three_rate=("PTS_TYPE", lambda values: (values == 3).mean()),
        )
        .reset_index()
    )
    return style[style["shots"] >= MIN_SHOTS].copy()

def label_archetypes(profiles: pd.DataFrame) -> dict[int, str]:
    """Give each cluster a descriptive label based on its centroid profile.
```

```

Labels are assigned by relative ranking rather than fixed thresholds so the
descriptions stay sensible even if the underlying numbers move a little:
the closest-to-the-rim cluster is the interior group, the most ball-dominant
of the rest are the creators, the most three-heavy of the rest are the
catch-and-shoot group, and the remaining cluster is the mid-range group.
"""
remaining = list(profiles.index)
labels: dict[int, str] = {}

interior = profiles.loc[remaining, "avg_shot_dist"].idxmin()
labels[interior] = "Interior Finishers"
remaining.remove(interior)

creators = profiles.loc[remaining, "avg_dribbles"].idxmax()
labels[creators] = "On-Ball Creators"
remaining.remove(creators)

shooters = profiles.loc[remaining, "three_rate"].idxmax()
labels[shooters] = "Catch-and-Shoot Specialists"
remaining.remove(shooters)

for cluster in remaining:
    labels[cluster] = "Mid-Range Scorers"

return labels

def build_archetypes(shots: pd.DataFrame, player_residuals: pd.DataFrame) -> pd.DataFrame:
    """Cluster the overperforming players into playing-style archetypes."""
    style = build_player_style(shots)
    style = style.merge(
        player_residuals[[PLAYER_NAME_COLUMN, "makes_above_expected"]],
        on=PLAYER_NAME_COLUMN,
        how="inner",
    )
    overperformers = style[style["makes_above_expected"] > 0].copy()

    scaled = StandardScaler().fit_transform(overperformers[STYLE_FEATURES])
    model = KMeans(n_clusters=N_ARCHETYPES, random_state=42, n_init=10)
    overperformers["cluster"] = model.fit_predict(scaled)

    profiles = overperformers.groupby("cluster")[STYLE_FEATURES].mean()
    overperformers["archetype"] = overperformers["cluster"].map(label_archetypes(profiles))

    return overperformers.sort_values(
        ["archetype", "makes_above_expected"], ascending=[True, False]
    ).reset_index(drop=True)

def build_rank_correlations(rank_table: pd.DataFrame) -> pd.DataFrame:
    """Spearman correlation between the adjusted metric and the traditional ones."""
    rows = []
    for metric in ["fg_pct", "efg_pct"]:
        rho, p_value = spearmanr(rank_table["shot_quality_adjusted"], rank_table[metric])
        rows.append({
            "comparison": f"shot_quality_adjusted vs {metric}",
            "spearman_rho": rho,
            "p_value": p_value,
        })
    return pd.DataFrame(rows)

def main() -> None:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

```

```

shots = build_modeling_frame(load_kaggle_shot_logs())
player_residuals = pd.read_csv(OUTPUT_DIR / "player_residuals.csv")
rank_table = pd.read_csv(OUTPUT_DIR / "rank_table.csv")

archetypes = build_archetypes(shots, player_residuals)
correlations = build_rank_correlations(rank_table)

archetypes.to_csv(OUTPUT_DIR / "player_archetypes.csv", index=False)
correlations.to_csv(OUTPUT_DIR / "rank_correlations.csv", index=False)

print("Player archetypes:")
print(archetypes["archetype"].value_counts().to_string())
print("\nRank correlations:")
print(correlations.to_string(index=False))
print(f"\nWrote player_archetypes.csv and rank_correlations.csv to "
      f"{OUTPUT_DIR.relative_to(REPO_ROOT)}")

if __name__ == "__main__":
    main()

```